



Testing reinforcement learning systems: A comprehensive review[☆]

Amal Sunba^{a,b}, Jameleddine Hassine^{a,c},*, Moataz Ahmed^{a,d}

^a Information and Computer Science Department, KFUPM, Dhahran, 31261, Saudi Arabia

^b Department of Computer Sciences, Princess Nourah Bint Abdulrahman University, Riyadh, 11671, Saudi Arabia

^c Interdisciplinary Research Center for Intelligent Secure Systems, KFUPM, Dhahran, 31261, Saudi Arabia

^d SDAIA-KFUPM Joint Research Center for Artificial Intelligence, KFUPM, Dhahran, 31261, Saudi Arabia

ARTICLE INFO

Keywords:

Software testing
Testing reinforcement learning
Review
Verification
Validation

ABSTRACT

Reinforcement Learning (RL) enables autonomous decision-making in dynamic environments, making it suited for complex, high-stakes domains like healthcare and defense systems. However, RL's high dimensionality and non-deterministic behavior pose testing challenges. This study presents the first literature review on testing RL systems, analyzing 49 studies published between 2013 and May 2025. The review categorizes testing RL techniques based on key workflow components: testing objectives, test generation, test oracles, and test adequacy. It identifies eleven primary gaps, including the lack of validation for testing RL frameworks in real-world applications and the need for specialized testing to verify RL-specific objectives, such as fairness and generalization. Additionally, the review highlights four key challenges: stochasticity leading to inconsistent fault detection, scalability and efficiency constraints in testing adequacy, fault identification complexity due to RL-specific failure definitions, and validation limitations due to reliance on simple tasks and underdeveloped test oracles. Our analysis shows that current research focuses on single-agent RL, robustness, and safety, yet these areas still contain gaps that require further exploration. The findings highlight that testing RL has become an active research area, peaking in 2023 and 2024, with 57% of the reviewed papers published these years. The identified challenges and gaps present opportunities for future research, guiding efforts toward more comprehensive and effective methodologies for testing RL.

Contents

1. Introduction	2
2. Background and foundations of testing RL	3
2.1. Fundamentals of reinforcement learning	3
2.2. Testing RL systems	3
2.2.1. Test input generation	4
2.2.2. Test oracle	6
2.2.3. Test adequacy	6
2.2.4. RL testing levels, methods, and goals	7
2.2.5. Test prioritization	7
2.2.6. Fault taxonomy in testing RL	8
2.2.7. Debug and repair	8
2.2.8. General testing framework and tools	8
2.2.9. Tested RL objectives	8
2.2.10. Testing components in RL	9
2.3. Related reviews	10
3. Research methodology	10
3.1. Research questions	10
3.2. Search strategy	10
3.2.1. Search terms	10

[☆] Editor: Torvald Mårtensson.

* Correspondence to: KFUPM, Dhahran 31261, Saudi Arabia.

E-mail address: jhassine@kfupm.edu.sa (J. Hassine).

3.2.2.	Literature resources	11
3.2.3.	Search selection process	11
3.2.4.	Study selection	11
3.2.5.	Data extraction	12
4.	Results and observations on RL testing research	12
4.1.	Distribution of research focus across RL testing aspects	12
4.2.	Evolution of RL testing research	13
4.3.	Peer review status and publication venues in RL testing research	14
4.4.	Citation evolution	14
5.	Research questions: Findings and discussion	14
5.1.	RQ1. What are the main testing objectives in RL research, and how do the testing approaches vary based on these objectives?	15
5.2.	RQ2. Which components of RL systems are typically tested, and what testing methods (e.g., fuzzing, mutation, model-based, simulation-based) are used to assess key testing objectives like robustness, safety, and effectiveness?	18
5.3.	RQ3. How do agent types and environment complexity influence testing strategies and objectives	20
5.4.	RQ4. What metrics evaluate test adequacy in testing RL systems?	20
5.5.	RQ5. What are the current research's main challenges and limitations in testing RL systems?	20
5.6.	Rq6. what research gaps exist in testing RL systems?	22
5.7.	Rq7. what are the potential future research directions for enhancing testing RL systems?	24
6.	Threats to validity	25
7.	Conclusions	25
	CRedit authorship contribution statement	26
	Declaration of Generative AI and AI-assisted technologies in the writing process	26
	Declaration of competing interest	26
	Acknowledgments	26
	Data availability	26
	References	26

1. Introduction

Software testing is a critical activity within the software development lifecycle. It ensures the final product meets user requirements, performs reliably under diverse conditions, and adheres to stringent quality standards. William Hetzel defined software testing as “any activity aimed at evaluating an attribute or capability of a program or system and determining that it meets its required results” (Hetzel, 1988). Testing is the measurement of software quality. According to the ANSI/IEEE Standard 729–1983, testing is “the process of exercising or evaluating a system or system component by manual or automated means to verify that it satisfies specified requirements or to identify differences between expected and actual results” (IEEE, 1983). Boris Beizer argues that tests are designed to ensure that software aligns with its specifications (Beizer, 1990). Glenford J. Myers defines testing as “the process of executing a program with the intent of finding errors” (Myers, 2004). These definitions collectively highlight the role of software testing in identifying defects that contradict software requirements.

Reinforcement Learning (RL) is a key AI approach and a subset of Machine Learning (ML) that enables systems to learn optimal behaviors through trial and error without requiring extensive labeled datasets (Sutton and Barto, 1998). Unlike traditional ML (especially in supervised learning), which relies on fixed datasets with labeled ground truth, RL agents learn from interactions with their environment, optimizing decisions based on state–action pairs and rewards. RL is effective for applications where conditions change unpredictably.

RL systems operate autonomously, optimizing resource usage and improving efficiency through continuous learning (Sutton and Barto, 1998; Che et al., 2020). By evaluating potential outcomes, they enable better decision-making, especially in real-time environments (Daoun et al., 2021; Reuer et al., 2023). RL excels in solving complex, multi-dimensional problems that challenge traditional algorithms (Landers and Doryab, 2023). These capabilities make RL essential across various domains. In healthcare, it optimizes personalized treatment plans (Liu et al., 2022), supports robotic surgery (Marijan and Gotlieb, 2020), and enhances medical image diagnostics (Hu et al., 2022). In military applications, RL aids in drone navigation (Wang and Wang, 2024),

autonomous vehicle mission planning (Kiran et al., 2022), and real-time strategy development during operations (Dimitriu et al., 2024).

Due to their complexity and deployment in high-stakes environments, RL systems require rigorous testing to verify both functional and non-functional requirements. Defining bugs is challenging, as failures often emerge over entire episodes, and ground truth is typically unavailable. Testing focuses on avoidable faults in the learned policy rather than unavoidable variability.

Research on testing RL has grown since 2013, peaking in 2023–2024. Existing reviews note a lack of approaches tailored to RL’s unique challenges. Landers and Doryab focused on DRL verification, highlighting issues like scalability and multi-agent complexity (Landers and Doryab, 2023). Nayab and Wotawa found limited work on testing RL despite its growing adoption (Nayab and Wotawa, 2024). However, these surveys do not comprehensively examine the methodologies, objectives, or components of RL testing. Further details are provided in Section 2.3.

This study investigates in-depth testing of RL, focusing on testing practices and challenges arising from testing RL itself and the nature of RL systems affecting their testability. This study presents the first comprehensive review of testing RL systems, providing the following contributions:

- An overview of 49 studies (2013–May 2025) on testing RL, covering key workflow aspects such as objectives, test generation, adequacy, and fault analysis.
- A structured analysis addressing the seven research questions (RQs) outlined in Section 3.1, covering:
 - Testing objectives by purpose, such as robustness and safety, with a focus on approaches and metrics.
 - Test generation techniques, such as adversarial, mutation, fuzzing, and search-based methods, and their impact on test effectiveness.
 - Fundamental types categorize item Targeted RL systems: model-free/model-based, single/multi-agent, and simulated/real environments.
- Identification of eleven key research gaps, including:
 - **Testing objectives:** Insufficient testing of RL systems for transparency, fairness, and scalability.

- **RL system types:** Limited testing approaches for model-based RL and large-scale systems, with minimal focus on multi-agent RL.
- **Validation of Testing Frameworks:** Lack of standardized benchmarks for RL testing and inadequate validation using real-world applications.

The remainder of this paper is organized as follows. Section 2 provides the background and foundational concepts relevant to testing reinforcement learning (RL) systems. Readers already familiar with the topic may choose to skip this section. Section 3 presents the research questions and details the approach used for search, data extraction, and synthesis. Section 4 presents a meta-analysis of the existing literature on RL testing. Section 5 addresses the research questions by discussing key findings, highlighting research gaps, proposing strategies to address them, and outlining future research opportunities. Threats to validity and corresponding mitigation strategies are discussed in Section 6. Finally, Section 7 outlines the main conclusions and their implications for future research.

2. Background and foundations of testing RL

This section provides the necessary background for understanding the challenges and methodologies involved in testing RL systems. It begins with an overview of fundamental RL concepts, then introduces a structured framework for categorizing testing approaches, and finally situates our work within the context of related survey literature.

2.1. Fundamentals of reinforcement learning

This section introduces and defines essential RL terminology, making the survey self-contained and accessible to readers unfamiliar with these concepts.

- **Learning Through Interaction: The RL Paradigm:** The foundation of RL lies in learning through interaction with an environment. RL explores idealized scenarios to test learning methods and enable autonomous decision-making through experience. As defined by Sutton and Barto, RL focuses on closed-loop problems in which an agent maps situations to actions to maximize cumulative rewards (Sutton and Barto, 1998). Unlike supervised learning, which relies on labeled data, RL agents learn through trial and error, where each action alters the environment and influences future states. Feedback in the form of rewards or penalties guides decision-making, helping agents optimize long-term cumulative rewards (Rojas et al., 2024).
- **RL Core Components:** The core elements of an RL system include the agent and its environment, with four key components: the policy, reward signal, value function, and, optionally, a model of the environment. The policy maps states or observations to actions, defining the agent's behavior. The reward signal provides immediate feedback, guiding the agent to maximize cumulative rewards. Rewards reflect short-term outcomes. The value function estimates long-term benefits to improve decision-making. States and observations represent the environmental conditions the agent perceives when acting.
- **RL Learning Strategies:** RL techniques differ by the type of learning approach. For example, Temporal Difference (TD) learning, a key RL approach, updates value estimates based on discrepancies between predicted and actual rewards. TD methods can be either on-policy or off-policy. On-policy methods, such as SARSA (Zou et al., 2019) and policy gradient (Lehmann, 2024), evaluate actions based on the current policy. Off-policy methods, like Q-learning (Jang et al., 2019) and Deep Q-Networks (DQN) (Lehmann, 2024), learn about a target policy using data from a different behavior policy.

- **RL Environment Types and Complexity:** RL environments are classified according to their determinism, observability, and complexity, influencing agent learning. Deterministic environments produce consistent outcomes for the same action, making transitions predictable. Stochastic environments introduce variability, requiring agents to repeat actions to identify reliable outcome patterns. RL environments vary by complexity and objectives, ranging from simulated to real-world settings. Simulated environments offer controlled, low-cost data, making them ideal for developing and testing RL models. Toy environments, such as those in OpenAI's Gymnasium, provide minimal complexity for debugging and validation (FaramaFoundation, 2024). Real-world environments introduce challenges like non-stationarity, high-dimensional interactions, and continuous action spaces (e.g., speed adjustments (Dulac-Arnold et al., 2021)). RL environments can be fully or partially observable. In fully observable environments, agents have complete visibility of the current state, enabling decisions based on direct information. Partially observable environments restrict visibility, requiring agents to rely on memory or inference (Muskardin et al., 2023). RL environments vary in state and action spaces, influencing the choice of learning methods. Traditional RL excels in structured, low-dimensional settings like cart-pole balancing or grid-based navigation, where states map to actions. Deep Reinforcement Learning (DRL) extends RL's capabilities to high-dimensional, unstructured environments by leveraging neural networks to process complex inputs (e.g., image pixels, sensor data), enabling applications like autonomous driving. RL environments can be categorized into three types: (1) limited states with discrete actions, like board games, suited to algorithms such as Q-learning. (2) unlimited states with discrete actions, where DQN can manage larger state spaces. (3) unlimited states with continuous actions, found in dynamic scenarios like UAV navigation, where policy gradient methods like PPO or DDPG are preferred (AlMahamid and Grolinger, 2022).
- **Agent Types in RL Systems:** Agents in RL systems can be deterministic or stochastic, on or off-policy, and single or multi-agent. Deterministic agents follow a fixed policy. Stochastic agents act probabilistically. On-policy agents learn from actions taken under the current policy. Off-policy agents learn from actions generated by different policies (Sutton and Barto, 1998). Single-agent RL optimizes rewards without competing influences. Multi-agent RL introduces multiple agents within the same environment, requiring cooperation for shared goals or competition (Hoen et al., 2005).
- **Model-Based vs. Model-Free Paradigms:** RL approaches are categorized as model-based or model-free. Model-based methods use an internal environment model to predict outcomes, enabling planning and simulation. Model-free methods, in contrast, learn optimal behaviors through experience (trial and error). Model-free approaches are less effective in complex environments (Morerland et al., 2023; Sutton and Barto, 1998).

Together, these foundational RL concepts form the basis for our exploration. This survey examines testing approaches proposed for all RL environments, agent types, and learning paradigms discussed above (see Fig. 1).

2.2. Testing RL systems

This section organizes the research on testing RL systems based on the taxonomy shown in Fig. 2. To construct this taxonomy, we first clarified the objective of organizing key aspects of RL testing. We then reviewed the literature that focused on testing practices, techniques, and evaluation in reinforcement learning. Relevant studies were selected based on their contributions to testing workflows, properties being tested, and specific components under evaluation (the first level

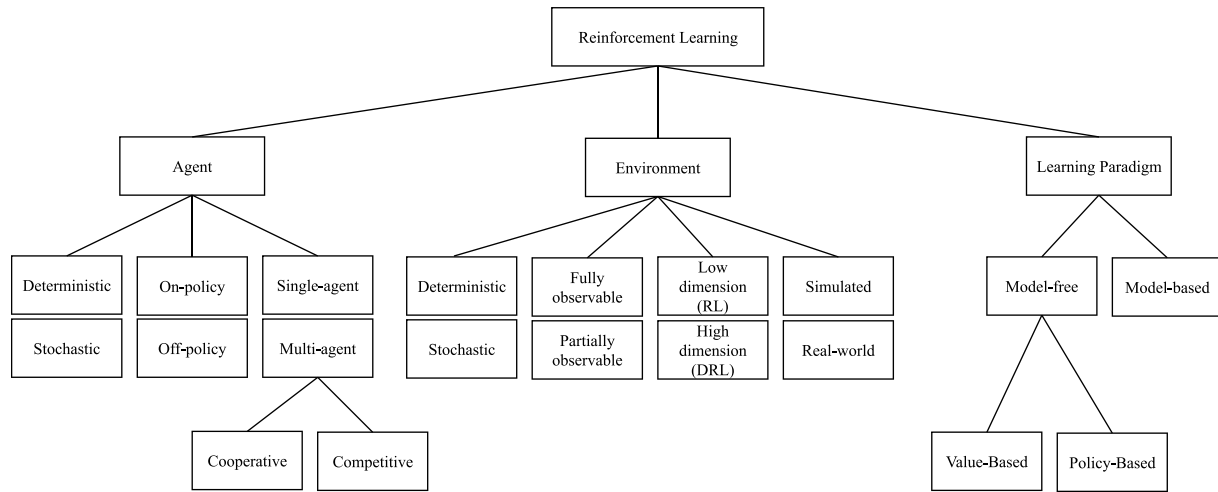


Fig. 1. Pillars of reinforcement learning.

Source: This conceptual overview is adapted from Sutton and Barto (1998), Rojas et al. (2024), Dulac-Arnold et al. (2021), AlMahamid and Grolinger (2022), Muskardin et al. (2023), Hoen et al. (2005) and Moerland et al. (2023).

of the taxonomy in Fig. 2). From these, we extracted recurring concepts such as test generation methods, fault detection strategies, test adequacy metrics, and types of coverage (the second level of the taxonomy in Fig. 2). The structure was refined iteratively, with subcategories added to capture method variants (e.g., adversarial vs. search-based testing), oracle types (e.g., metamorphic or fuzzy compliance), and test adequacy criteria (e.g., neuron coverage, mutation score).

2.2.1. Test input generation

We categorize test input generation techniques based on the applied techniques.

- **Adversarial Testing:** Uesato et al. introduced the AVF, a failure probability predictor designed to improve the evaluation process of RL agents by estimating the likelihood of catastrophic failures based on initial conditions (Uesato et al., 2019). It leverages data from less robust agents to enhance the learning signal for predicting failures, concentrating on adversarial scenarios and utilizing historical data. Furthermore, Casper et al. proposed a white-box adversarial testing methodology for RL agents that leverages the target agent's internal state information to generate adaptive adversarial policies (Casper et al., 2023). This approach creates adversarial policies through PPO that dynamically adapt to the target's state and actions by accessing the agent's latent representations, action outputs, and value estimates.

Testing MARL system, Phan et al. proposed the Antagonist-Ratio Training Scheme (ARTS) to generate adversarial test scenarios in the MARL system (Phan et al., 2020). ARTS designates a portion of agents as antagonists based on a specified ratio during testing, creating varied failure scenarios in each episode. Guo et al. introduced MARLSafe, which applies state perturbations via gradient-based FGSM to mislead agents' decisions by reducing the probability of optimal actions (Guo et al., 2022). Reward perturbations are introduced by flipping a portion of reward signs during training, disrupting agents' learning and encouraging ineffective behavior. Action perturbations use adversarial policies to control one agent as a "traitor" that disrupts team coordination. These tests reveal weaknesses in agents' perception, learning, and cooperation under adversarial conditions. Zhou and Liu employed a differential evolution (DE) approach in RTCA to identify critical agents (victims) and generate worst-case joint actions, focusing

on those whose actions most impact team coordination (Zhou and Liu, 2023). The DE methodology optimizes perturbations to minimize cumulative team rewards, thus exposing vulnerabilities in the agents' cooperative policy. RTCA is flexible. It can vary the victim agent at each timestep, making the attack stealthier and more effective. Chen et al. introduced an adversarial testing framework (Demo2Test). It uses transfer learning to train a testing agent via RL and imitate failure demonstrations from a source task (Chen et al., 2025).

- **Fuzzing and Search-Based Testing:** Eniser et al. proposed π -fuzz, which incrementally builds a pool of diverse states by performing random walks in the environment (Eniser et al., 2022). Each random walk either chooses random actions or samples from the policy. After a walk, the resulting state is added to the pool if its Euclidean distance from already stored states exceeds a predefined threshold. Mazouni et al. investigated and replicated the MDPFuzz framework (Pang et al., 2022) for black-box MDP testing, targeting decision-making policies. MDPFuzz uses Gaussian Mixture Models (GMMs) to guide its coverage-based fuzzing, aiming to discover more faults. Mazouni et al. replicated this GMM-based approach and extended the evaluation to additional RL tasks (Mazouni et al., 2025). Their findings reveal that while coverage guidance can help in some scenarios, a simpler coverage-free variant often outperforms MDPFuzz in fault detection and efficiency. AgentFuzz, proposed by Li et al. generates diverse initial states, monitors each episode's total reward, and labels those below a threshold as failed. It remains a doctoral proposal without a detailed experimental evaluation (Li et al., 2022b). Expanding on these ideas, Wan et al. introduced DRLFuzz, a search-based approach that employs gradient ascent-based mutations to generate failure-inducing states. Starting from an initial state set, it mutates selected states along loss gradients while leveraging coverage feedback to maximize state-space exploration (Wan et al., 2024).

The integration of search and fuzzing techniques is successfully applied in Tappler et al. (2022), where a depth-first search generates a reference trace that solves the RL task and identifies boundary states, specific safety-critical states near potential failure states. This reference trace serves as the basis for test generation. The authors then applied fuzzing for a diverse test suite, using

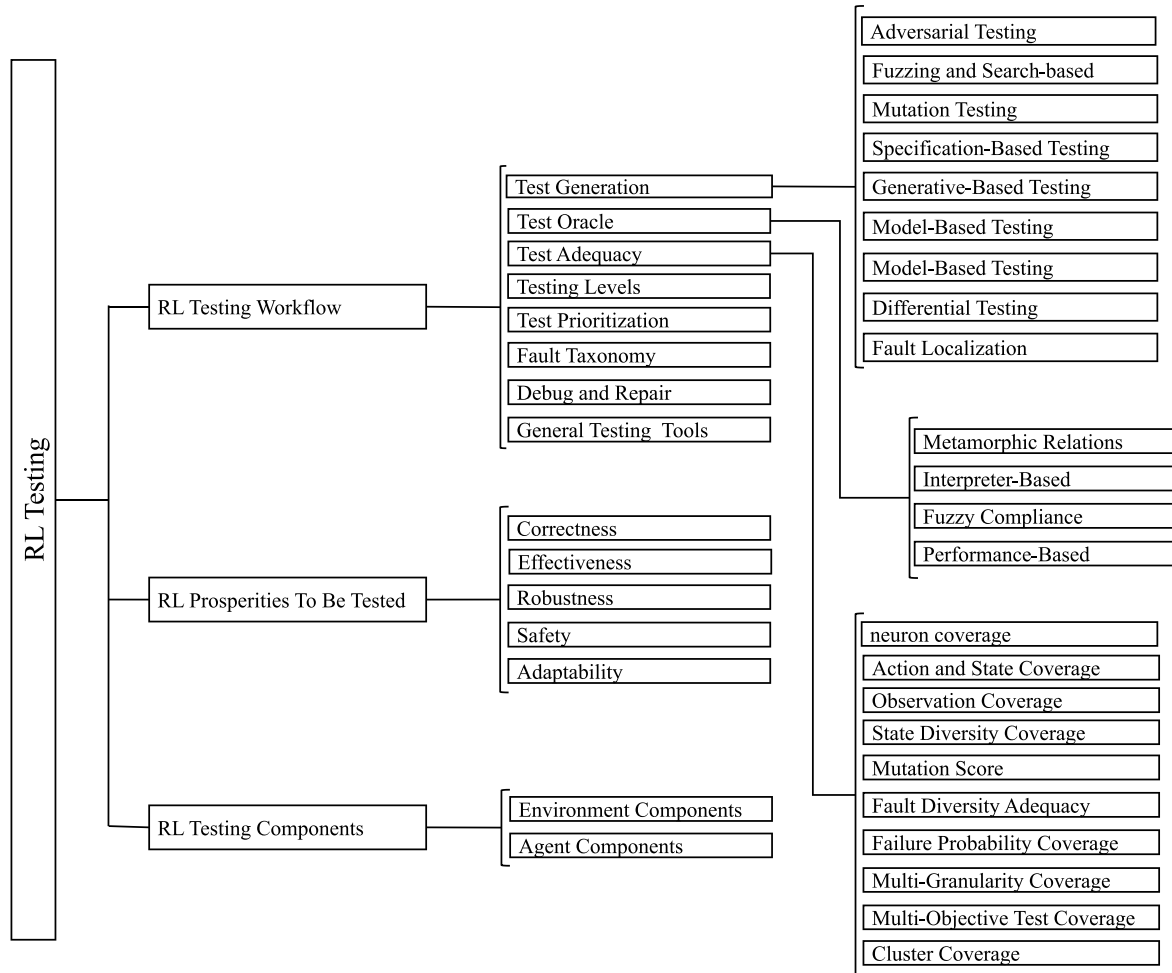


Fig. 2. Literature review schema.

genetic algorithms to mutate and crossover traces. This approach tests an agent's ability to maintain performance across diverse and challenging scenarios. Zolfagharian et al. proposed STARLA, a data-box search-based testing approach that employs genetic algorithms to detect functional faults in DRL agents. It evolves a population of episodes, using fitness criteria (e.g., low rewards or high fault probability) to guide the search toward potential failures (Zolfagharian et al., 2023). Tambon et al. applied heuristics to generate diverse test environments that optimize fault detection in DRL systems (Tambon et al., 2023). To explore the behavioral space of learned policies, Mazouni et al. introduced the use of Quality Diversity (QD) optimization for generating diverse test cases (Mazouni et al., 2024). The approach implements MAP-Elites and Novelty Search to derive test cases that uncover diverse faults in the RL system. Biagiola and Tonella proposed Indago, which trains a classifier on failure and non-failure configurations from the DRL training process as a surrogate model (Biagiola and Tonella, 2024). It uses failure predictions as a fitness function, employing hill climbing and genetic algorithms to generate diverse, failure-inducing environment configurations. He et al. proposed CureFuzz, which utilizes a novel curiosity mechanism that quantifies the novelty of scenarios encountered during testing (He et al., 2024). Ma et al. proposed MASTest, a framework that enhances test diversity in MARL using grid-based state abstraction to simplify complexity and compare behaviors (Ma et al., 2024). It evaluates individual and team diversity through agent trajectories and interactions, guiding exploration via diversity measurements.

- Mutation Testing:** Lu et al. assessed RL agent robustness by introducing controlled mutations at both the environment and agent levels, such as modifying state or reward data and altering the Q-table or neural network structure (Lu et al., 2022). This identifies potential bugs by comparing the mutated agent's performance to that of the original, with a high mutation score indicating strong bug detection. Tambon et al. (2023) introduced RLMutation, a mutation testing framework for RL that defines eleven targeted mutation operators across three levels: environment, agent, and policy, inspired by real faults. It uses First-Order Mutations (FOMs) to simulate individual faults and Higher-Order Mutations (HOMs) to combine multiple faults. Additionally, Thomas et al. proposed μ PRL, a mutation testing approach that implements mutation operators derived from a taxonomy of real RL faults identified through repository mining (Thomas et al., 2024).
- Specification-Based Testing:** Varshosaz et al. proposed a specification-based test generation framework for RL systems, focusing on verifying the correctness of temporal difference methods like SARSA and Q-learning (Varshosaz et al., 2023). The framework employs a property-based test that generates tests from formal RL specifications. Key tested properties include state observability, termination conditions, and convergence to ensure reward estimates remain within bounds. It offers both generic and algorithm-specific tests to validate state-action updates.
- Generative-Based Testing:** Li et al. introduced a generative-based test generation framework for decision-making policies, leveraging a diffusion model to create diverse test cases. The

diffusion model is trained on random inputs to capture the distribution of valid cases. Then, in the fine-tuning stage, the model applies a novelty-based guide mechanism, enhancing test case diversity by targeting unexplored or vulnerable areas in the agent's decision space, particularly termination states (Li et al., 2023).

- **Simulation-Based Testing:** Allen et al. introduced a simulation-based testing approach for optimizing ambulance location assignments using DRL (Allen et al., 2021). They developed an OpenAI Gym-compatible simulation environment to test DQN-based RL agents in a simplified ambulance dispatch model. This environment allows repeated testing to minimize response times and control incident distribution.
- **Model-Based Testing:** Nikanjam et al. introduced DRLinter, a testing tool for detecting faults in DRL programs (Nikanjam et al., 2022). It uses a meta-model that defines the key components of DRL systems, such as agent-environment interaction and decision-making processes. DRLinter parses the source code into a graph that adheres to this meta-model, enabling static analysis. The tool identifies faults related to structural inconsistencies and model violations by applying graph transformation rules. Tappler et al. proposed a testing method that learns an abstract MDP from the execution traces of a trained DRL agent. This model serves as a basis for generating test inputs. Three strategies are employed for test case selection: random goals, equidistant goals from the start state, and branching point goals, target locations in the MDP where many alternative transitions exist (Tappler and Lorber, 2024).
- **Automata Learning and Differential Testing:** Tappler and Aichernig introduced black-box testing, combining automata learning and differential testing. The IOAlergia algorithm learns abstract models of the environment by collecting data from executed episodes to create observation traces. These traces are then abstracted into a deterministic labeled MDP. Once the abstract model is created, control policies are derived to guide agents to specific states for testing. Test-case specifications define the abstract observations that need to be covered. Differential testing is applied to compare the behaviors of different RL policies under similar conditions (Tappler and Aichernig, 2023). Tappler et al. introduced CASTLE, another framework that employs automata learning to model environmental behaviors in RL systems. By leveraging dimensionality reduction and clustering, CASTLE handles complex, continuous environments, enabling efficient testing of deep RL agents. The framework operates in two phases: first, learning an initial model from observed agent-environment interactions, and second, refining the model through active sampling. CASTLE's approach allows precise test generation without requiring pre-existing environmental models, effectively bridging model-free RL agent training with testing mechanisms (Tappler et al., 2024a).
- **Statistical-based Testing** Li et al. tested stationarity and detected change points in offline RL systems. Their model assesses the consistency of the optimal Q-function over time and integrates change point detection for better policy optimization in nonstationary environments. Li et al. (2022a). AdaStop applies a sequential, nonparametric approach to compare RL agents with stochastic outcomes. The method controls error rates and adaptively determines the executions needed to distinguish agent performance, leveraging historical data to refine testing. It is computationally efficient, stopping early when sufficient evidence is gathered (Mathieu et al., 2023).
- **Domain-Specific Applications Testing** Raz et al. introduced a three-part Test and Evaluation (T&E) framework for RL in aerospace applications. The framework comprises robustness testing, SHAP explainability, and comparison with traditional optimal control. In the robustness testing component, RL performance is evaluated by varying initial conditions like altitude and velocity

to assess sensitivity and performance boundaries. This testing helps characterize the RL agent's response to environmental uncertainties. The explainability component uses SHAP to analyze decision-making, highlighting key features like altitude that influence RL actions. RL outputs are compared to optimal control trajectories to validate the agent's decisions within acceptable performance bounds (Raz et al., 2022).

- **Fault Localization** Moran et al. proposed SBFL4RL, a fault localization approach for RL that automatically identifies states leading to poor performance by analyzing episode outcomes. It ranks states based on their coverage in low-return episodes compared to high-return ones, assuming frequently visited states during low-performing episodes are more likely to be faulty (Morán et al., 2023).

Table 1 summarizes the components each approach utilizes for test suite generation.

2.2.2. Test oracle

A test oracle serves as a reference point for evaluating RL agents. Establishing a test oracle is complex because RL systems are dynamic and adaptive. This section introduces test oracles explicitly mentioned or proposed in the collected papers:

- **Metamorphic Relations:** Eniser et al. proposed a test oracle for RL policies based on metamorphic testing with state relaxations. The oracle detects bugs by comparing the policy's performance on pairs of states where one is a relaxed, simpler version of the other. If the policy succeeds in the complex state but fails in the simpler one, it indicates a bug, as handling the relaxed state should be expected (Eniser et al., 2022).
- **Interpreter-Based:** Varshosaz et al. leveraged an interpreter for RL update equations, aligning agent behavior with a predefined formal model. This interpreter validates that each operation, such as state-action updates and episode terminations, conforms to the expected outcomes specified by the model (Varshosaz et al., 2023).
- **Fuzzy Compliance:** The test oracle proposed in Zhang et al. (2024) utilized fuzzy logic to tackle the complexities of RL programs. It generates a set of intended policies, each mapping selected states to corresponding ideal actions. The oracle evaluates an agent's compliance with these policies through a fuzzy reward function, quantifying how closely the agent's actions align with the intended actions based on fuzzy membership values. The oracle identifies suspicious behavior and determines if the program is buggy by calculating policy-compliance values over training episodes.
- **Performance-Based Test Oracle:** A performance-based test oracle in RL evaluates an agent's effectiveness using performance metrics across episodes. In the SBFL4RL, episodes are categorized by their return, with high-return episodes deemed successful and low-return episodes indicating potential faults (Morán et al., 2023). MDPFuzz assesses performance by detecting abnormal states, such as collisions or falls (Pang et al., 2022). Other methods establish specific failure criteria based on observable metrics, such as Cart Pole threshold violations, Lunar Lander crashes, or invalid actions in the Taxi environment (Mazouni et al., 2025, 2024).

2.2.3. Test adequacy

Test adequacy measures how effectively test cases evaluate a system. This section reviews test adequacy approaches and coverage criteria identified in our research.

- **Neuron Coverage:** Trujillo et al. investigated neuron coverage (NC) as a metric for assessing the effectiveness of DRL systems through white-box testing, analyzing the relationship between NC

Table 1
RL components utilized in each test generation approach.

Component \ Approach	Agent	Traces	Policy	Reward	State	Episode	Action	Env. Params	Observations	Test cases
Adversary training	✓									
Fuzzing		✓			✓	✓				✓
Mutation			✓	✓	✓					✓
Genetic operation						✓				
Adversarial perturbations				✓	✓		✓		✓	
Scaling								✓		
Metamorphic relation					✓					

and cumulative rewards. They examined NC evolution at both system and layer levels. The experiments revealed that increased NC does not correlate positively with reward in DRL (Trujillo et al., 2020). Tappler et al. evaluated NC as a test adequacy metric for DRL policies. The experiments showed that higher NC does not consistently correspond to more failure-detecting test cases and that NC does not reliably distinguish between agents trained to different levels of competence. They conclude neuron coverage is not a reliable proxy for test effectiveness in planning-based DRL (Tappler and Lorber, 2024).

- **Action and State Coverage:** Tappler et al. achieved comprehensive action and state coverage by combining depth-first search (DFS) and genetic algorithms. DFS identifies a reference trace that accomplishes the RL task while exposing critical boundary states. Genetic algorithms then generate diverse action traces by mutating and recombining these Ref. Tappler et al. (2022).
- **Observation Coverage:** Tappler and Aichernig used abstract observations from learned models to define test-case specifications. Coverage is measured by tracking which abstract observations are reached during test execution (Tappler and Aichernig, 2023).
- **State Diversity Coverage:** Li et al. used state diversity to evaluate the breadth of the agent's decision space explored during testing (Li et al., 2023). This method groups similar states in a cluster and tracks unique clusters encountered across test trajectories. Higher state diversity indicates broader exploration. Similarly, DRLFuzz employs an ANN-based coverage analyzer to track unique state regions visited by test cases (Wan et al., 2024). MDPFuzz incorporates Gaussian Mixture Models (GMMs) to guide coverage and explore underrepresented states (Pang et al., 2022).
- **Mutation Score:** Lu et al. and Varshosaz et al. both employ mutation scores to quantify the proportion of killed mutants and, consequently, the effectiveness of the test suite in detecting potential bugs. In Lu et al.'s study, the mutation score reveals the ability to detect distinct behaviors arising from introduced bugs. Varshosaz et al. demonstrate that a high mutation score, with up to 90% of mutants killed, indicates strong test coverage and the utility of tests in detecting faults (Lu et al., 2022; Varshosaz et al., 2023).
- **Fault Diversity Adequacy:** Mazouni et al. measured the diversity of failure states and distinct behaviors within the defined behavioral space, ensuring comprehensive exploration of potential failure modes (Mazouni et al., 2024).
- **Failure Probability Based Coverage:** The AVF approach covers initial conditions likely to result in catastrophic failures in RL agents. It estimates the likelihood of failure based on these initial conditions (Uesato et al., 2019).
- **Multi-Granularity Coverage:** Shi et al. measured the number of unique states visited, ensuring a diverse assessment. Action coverage ensures all actions are utilized. Reward coverage evaluates the distribution of received rewards across scenarios. Combinatorial coverage assesses interactions between various state-action pairs (Shi et al., 2024).

- **Multi-Objective Test Coverage:** CureFuzz ensures test adequacy by balancing novelty, crash probability, and robustness. The coverage is assessed through their potential to trigger crashes and explore diverse state (He et al., 2024).
- **Cluster coverage:** The CASTLE framework focuses on significant state clusters within the environment, ensuring testing targets key state areas. The effectiveness of this approach depends on the accuracy of the definition of state features in creating clusters (Tappler et al., 2024a).

2.2.4. RL testing levels, methods, and goals

The testing levels (unit, integration, system) are rarely mentioned in testing RL literature. Based on our review, we infer that system-level testing is predominant. This stems from the need to assess agent behavior in the context of its whole environment interaction, which is essential for evaluating performance, robustness, and multi-agent RL coordination. Although components like reward functions, policy networks, or dynamics models could be tested at the unit or integration level, our review found no studies applied such practices. Fault taxonomies by Nikanjam et al. and Thomas et al. include issues detectable at these levels (Nikanjam et al., 2022; Thomas et al., 2024).

Dynamic testing is the focus of current RL testing work. Methods such as adversarial perturbation, fuzzing, and mutation testing evaluate agent behavior during execution. Conversely, static testing approaches remain limited but notable. DRLinter statically analyzes DRL programs using graph transformation rules over a meta-model extracted from source code. It identifies structural and configuration faults without executing the code (Nikanjam et al., 2022). The MOSAIC framework applies symbolic static analysis through abstract interpretation to verify the safety properties of DRL controllers (Bacci and Parker, 2020). Additionally, Li et al. proposed a statistical static testing approach to detect nonstationarity in offline RL by analyzing Q-function behavior over time (Li et al., 2022a). Studies have used faults found during testing to repair and improve RL systems. These repaired systems are then re-evaluated to ensure performance is maintained, which aligns with the principles of regression testing, e.g., Tappler et al. (2024b), Wan et al. (2024), Pang et al. (2022), Mazouni et al. (2025) and Chen et al. (2025).

2.2.5. Test prioritization

Prioritization optimizes testing by ranking test cases based on their potential to uncover bugs. It improves efficiency by reducing the computational costs of the test.

MASTest employs a state criticality table to prioritize testing efforts by evaluating each state's potential to trigger diverse failures. This table records the perturbation potential of each state based on three aspects: diversity gain, failure gain, and selection frequency. A state with a high potential score is considered critical and should be prioritized for perturbation. The criticality of states is dynamically updated after each test run (Ma et al., 2024). CureFuzz and MDPFuzz adopt energy-based seed prioritization, where seeds with higher energy (estimated

by their sensitivity to triggering crashes or novel behaviors) are priority in selection (He et al., 2024; Pang et al., 2022). Tappler et al. introduced a branching point-based prioritization strategy within an abstract MDP, where states with high branching factors are selected as test goals (Tappler and Lorber, 2024). IMT prioritizes testing by ranking states according to their importance for policy safety, focusing on the most critical states, enabling early discovery of unsafe behavior while reducing verification cost (Pranger et al., 2024).

2.2.6. Fault taxonomy in testing RL

Nikanjam et al. presented a taxonomy of faults occurring in DRL programs based on analyzing 761 artifacts from GitHub and Stack Overflow. They identified eleven fault categories, focusing on DRL-specific issues related to the interaction between agents and environments and the implementation of DRL algorithms (Nikanjam et al., 2022). Thomas et al. introduced a fault taxonomy for RL systems based on 2787 posts from GitHub and Stack Overflow. It includes faults in reward functions, training, normalization, inference, environment setup, state representation, network updates, and exploration (Thomas et al., 2024). STARLA utilizes classifiers to characterize and explain failed test cases, enhancing fault analysis by distinguishing between policy failures and environment-related issues (Zolfagharian et al., 2023). Li et al. introduced 107 domain-specific mutation operators for testing DRL that simulate faults across agent training and neural policy structures (Li et al., 2025).

2.2.7. Debug and repair

Li et al. retrained models with bug-triggering trajectories identified through testing. Policies in two environments, the Bipedal Walker simulated walking task and ACAS Xu aircraft collision avoidance system, were fine-tuned accordingly. Retraining with these detected bugs reduces errors significantly (Li et al., 2023). DRLFuzz supports system repair by fine-tuning the DRL agent using generated failed cases (Wan et al., 2024). These cases serve as starting points for retraining the agent with a small learning rate, effectively addressing failures caused by insufficient exploration during initial training. Tappler et al. proposed a policy repair method that leverages safe demonstrations generated from fuzzing-based safety tests. When unsafe behavior is detected, the approach identifies repair states and extracts relevant safe experiences from the demonstrations. These update the agent's policy via a modified DQN, enabling safe recovery in problematic states while preserving overall performance. The process is repeated until safety and performance goals are met (Tappler et al., 2024b). Ma et al. incorporated the failure scenarios identified during testing into the repair process, significantly reducing collision occurrences by 45.83% and improving task completion rates by 45.42% (Ma et al., 2024). CureFuzz integrates identified crash-triggering scenarios into the repair process, reducing detected faults by 73% (He et al., 2024). MDPFuzz applies fine-tuning to the agent using failure-inducing inputs, improving post-repair robustness across multiple environments by reducing crash rates and enhancing policy reliability (Pang et al., 2022).

2.2.8. General testing framework and tools

Biagiola and Tonella introduced Indago, which employs surrogate models and search-based techniques to generate diverse, failure-inducing environment configurations for testing DRL agents (Biagiola and Tonella, 2024). The Arcade Learning Environment (ALE) is widely used for benchmarking RL agents across Atari 2600 games (Bellemare et al., 2013). It standardizes testing by converting games into RL problems where agents aim to maximize rewards. While ALE does not explicitly support fault detection, its performance evaluation across diverse games can highlight agent vulnerabilities or areas for improvement. ToyBox was introduced as a high-performance testing suite designed to enhance the evaluation of DRL agents (Tosch et al., 2019). It offers flexibility by allowing researchers to manipulate game states

and environments, which opens possibilities for post-training evaluations, dynamic testing during training, and counterfactual experiments. Both tools play roles in RL research. ToyBox builds on the foundation laid by ALE, offering more customization and dynamic testing options. ALE remains a widely accepted benchmark for domain-independent RL evaluation.

COOL-MC is a comprehensive tool that integrates RL and formal verification through model checking. It tests RL policies by extracting discrete-time models from learned behaviors and verifying them against temporal logic specifications using the Storm model checker. It supports state abstraction and remapping (Gross et al., 2022).

2.2.9. Tested RL objectives

An RL system must meet functional and non-functional requirements defined in its specifications. Testing primarily aims to identify bugs and flaws that hinder achieving these requirements. This section overviews studies that explicitly propose testing RL for specific objectives, detailing the approaches and evaluation metrics applied.

- **Correctness:** Ensures that RL agents produce appropriate behaviors or achieve task-specific objectives.
 - *Correctness Measurement Criteria:* Varshosaz et al. assessed correctness in RL systems using key criteria derived from formal specifications (Varshosaz et al., 2023). These criteria include verifying that each system state maps to an observable state and ensuring the RL agent reliably perceives all necessary states. Termination conditions are validated to guarantee proper episode completion; learning updates are monitored to prevent divergence errors.
 - *Correctness Testing Approaches:* Varshosaz et al. employed a specification-based framework to validate RL agents' correctness by comparing their behavior to a formal model (Varshosaz et al., 2023).
- **Reliability, Robustness and Resilience:** Reliability ensures consistent performance, robustness evaluates the system's ability to handle adversarial and unexpected conditions, and resilience assesses recovery and adaptation after disruptions. These objectives are often used interchangeably and may overlap in testing approaches.
 - *Reliability, Robustness and Resilience Measurement Criteria:* Phan et al. tested c-MARL resilience using two metrics: completion rate, which indicates how reliably tasks are completed under adversarial interference, and flaw probability ($1 - \text{completion rate}$), which captures the likelihood of failure under severe disruptions (Phan et al., 2020). Guo et al. tracked win rate, team reward, mean of dead allies number, and mean of enemies killed number for the robustness of c-MARL agents in the StarCraft challenge (Guo et al., 2022). Lu et al. used mutation score changes to measure robustness, observing shifts in scores when introducing bugs. Significant variations in mutation score signal potential bugs (Lu et al., 2022). Liu et al. evaluated the robustness of DRL algorithms by assessing their performance based on the percentage change in rewards resulting from the attacks (Liu et al., 2024). Zhou and Liu tracked changes in cumulative team rewards when adversarial perturbations were applied to critical agents within a c-MARL system. A significant drop in rewards indicates successful disruption of team coordination (Zhou and Liu, 2023). Behzadan and Hsu introduced adversarial regret, quantifying the difference in performance between the standard and perturbed policies (Behzadan and Hsu, 2019). Moreover, the AVF assesses the agent's reliability by estimating the failure probabilities associated with various initial conditions that could lead to catastrophic failures (Uesato et al., 2019).

- **Robustness and Resilience Testing Approaches:** Phan et al. proposed QMixMax for the extreme scenario of one protagonist agent versus $N - 1$ antagonists, leveraging an approximated Minimax-Q approach. By modeling this worst-case setup, QMixMax can expose potential vulnerabilities of the single target agent under severe adversarial conditions (Phan et al., 2020). MARLSafe employs adversarial attacks on the agents' multiple aspects (state, reward, and action) to observe how their performance degrades under adverse conditions (Guo et al., 2022). Lu et al. introduced reward and state-level mutations to simulate potential faults in RL systems. They evaluated system stability by observing the effects of these mutations and measuring changes in performance using mutation scores (Lu et al., 2022). Liu et al. subjected popular DRL algorithms to black-box and white-box adversarial attacks. Black-box attacks perturbed the observation and action channels, simulating environmental and malicious factors. White-box attacks used the Fast Gradient Sign Method and Projected Gradient Descent to assess algorithm performance under informed conditions (Liu et al., 2024). Casper et al. utilized adversarial evaluation, focusing on scenarios with a high probability of failure (Casper et al., 2023). Behzadan and Hsu simulated adversarial conditions to assess the resilience and robustness of DRL policies. They trained an adversarial agent to optimize disruption metrics against a target policy. In the testing phases, this adversarial policy is applied over multiple episodes to evaluate the impact of perturbations on the target policy (Behzadan and Hsu, 2019). Zhi et al. used reward martingales to certify the robustness of DRL policies. Their method estimates upper and lower bounds on cumulative rewards under bounded state perturbations (Zhi et al., 2024).
- **Safety:** It ensures that agents avoid unsafe states while achieving their objectives in safety-critical environments.
 - **Safety Measurement Criteria:** Corsi et al. introduced safe rate and violation rate for DRL policy evaluation. These metrics assess the proportion of input states where the policy satisfies or violates safety constraints (Corsi et al., 2020). Tappler et al. used safety fail frequency, which evaluates safety by tracking how often safety violations (fail verdicts) occur during testing. Fail verdict frequency measures the proportion of test cases where the agent enters unsafe states (Tappler et al., 2022). Tappler and Aichernig defined two types of terminal states: unsafe and goal states (Tappler and Aichernig, 2023).
 - **Safety Testing Approaches:** Bacci and Parker introduced the MOSAIC algorithm to evaluate the safety of DRL controllers in stochastic environments, using formal abstraction and model checking of MDPs to establish safety guarantees (Bacci and Parker, 2020). Mqirmi et al. introduced a formal verification method to ensure that DRL solutions in multi-agent settings satisfy safety constraints (Mqirmi et al., 2021). Gross and Spieker proposed a scalable and accurate formal safety verification method for stochastic DRL systems by analyzing discrete-time Markov chains induced by trained policies. It verifies memoryless policies against safety properties expressed in PCTL, improving over deterministic approximations. Gross and Spieker (2024). Zolfagharian et al. proposed SMARLA, a safety monitoring framework that predicts safety violations using ML based on Q-values and state abstraction. It focuses on runtime safety monitoring rather than pre-deployment testing (Zolfagharian et al., 2024). Tappler et al. identified boundary states during the DFS reference trace search. The framework brings the RL agent into these states to test safety and observe if it can avoid unsafe transitions. Testing evaluates the agent in three ways: simple boundary testing, which generates test cases ending at boundary states; boundary interval testing, which examines the agent's capacity to handle slight variations around boundary states; and action coverage testing, which introduces diverse action sequences near boundary states to evaluate whether different combinations of actions lead to unsafe outcomes (Tappler et al., 2022). Also, Tappler et al. introduced a probabilistic sampling mechanism during test execution, which randomly perturbs action selection, forcing the agent to encounter a broader range of scenarios (Tappler et al., 2024b).
- **Adaptability:** It focuses on the agent's ability to adjust and perform successfully in varying environments.
 - **Adaptability Measurement Criteria:** In AlphaTest, Biagiola and Tonella introduced Adaptation Probability and Regression Probability as core metrics for evaluating RL adaptability. Adaptation Probability quantifies the proportion of successful adaptations in a modified environment, measured across multiple trials with varied random seeds. Regression Probability assesses the likelihood of performance degradation in the original environment following adaptation, evaluated only after a successful adaptation. These metrics define an 'adaptation frontier', representing the boundary between environments where the agent succeeds or fails to adapt. Derived metrics, such as Adaptation Volume and Anti-Regression Volume, quantify the extent of adaptability and regression across the parameter space.
 - **Adaptability Testing Approaches:** The AlphaTest framework characterizes the adaptation capabilities of RL agents by leveraging exponential and binary search strategies to identify and refine the adaptation frontier. Once these boundary configurations are identified, AlphaTest modifies the environment parameters and evaluates whether the agent can adapt through continual learning. Adaptation success is defined when the agent achieves the desired performance in the modified environment, while failure occurs when performance falls below the required level (Biagiola and Tonella, 2022).

Table 2 summarizes the approaches utilized for each testing objective.

2.2.10. Testing components in RL

This section provides an overview of detecting faults in both the environment and the agent, highlighting their impact on system performance.

- **Bug Detection in Environment:** Nikanjam et al. identified faults in the interaction between DRL agents and environments. Key issues include the missing stepping of the environment, where the environment fails to update its state, leading to loss of reward tracking; the missing terminal state, where episode termination is not detected, resulting in inefficient learning; and the missing reset, where proper termination and initialization for new episodes are not ensured. Misconfigurations in the action space can hinder the agent's response (Nikanjam et al., 2022).
- **Bug Detection in Agent:** Nikanjam et al. identified faults in the agent's functionality that hinder performance, including wrong network updates, where improper adjustments to the agent's network destabilize learning, and suboptimal exploration rate, which reflects an ineffective balance between exploration and exploitation, impairing the agent's learning from the environment (Nikanjam et al., 2022). Regarding c-MARL, Guo et al.

Table 2
Testing approaches used for different objectives in RL.

Objective Approach	Resilience	Robustness	Safety	Performance	Adaptability	Correctness
Adversarial	✓	✓				
Fuzzing	✓	✓	✓	✓		
Research-based		✓	✓		✓	
Mutation		✓				✓
Formal specifications						✓
Formal-verification			✓			
Data-driven					✓	
Simulation-based						✓

applied adversarial testing to cause agents to take suboptimal actions, disrupt team coordination, and misguide agents to avoid engagement (Guo et al., 2022). Zhou and Liu exposed policy vulnerabilities by causing targeted agents to take disruptive actions by applying adversarial perturbations to agents' observations. The resulting miscoordination reveals flaws in the agents' learned cooperation strategies (Zhou and Liu, 2023). Casper et al. used white-box adversarial testing, where the adversary observes the agent's internal states, including latent representations and action values (Casper et al., 2023). Tambon et al. introduced mutations affecting policy functions and agent-level settings, exposing reward processing flaws and state updates impacting learning and adaptive behavior (Tambon et al., 2023). Zolfagharian et al. aimed to detect bugs in the RL agent's policy by identifying episodes where the agent's behavior is faulty (Zolfagharian et al., 2023). Eniser et al. employed relaxation-based metamorphic relations to uncover bugs in policy. If the policy succeeds in the original (harder) state but fails in the relaxed (easier) state, this indicates a bug in the policy (Eniser et al., 2022). Li et al. identified bugs in agent decision-making by pushing agents into unfamiliar states (Li et al., 2023).

2.3. Related reviews

Recently, there has been interest in RL, with papers reviewing various aspects of RL, including its algorithms, applications, and challenges. Despite the growing body of work, a lack of reviews dedicated to testing RL.

Reviews of software testing practices for ML systems discuss the unique testing needs of RL systems. Huang et al. emphasized the urgent need to address the safety and trustworthiness of RL models while noting that existing safety analysis techniques for DNNs may not adequately cover the complexities of RL (Huang et al., 2020). Zhang et al. conducted a thorough survey of ML testing techniques, identifying a gap in research on testing RL. Among 144 papers, only one explicitly addressed testing RL (Zhang et al., 2022). This scarcity reflects a broader imbalance in the literature on testing ML, where the focus has primarily been on supervised learning and classification problems. Landers and Doryab reviewed verification methods for DRL systems. Their survey covers verification methods such as specification-based and abstraction-based approaches and discusses properties that need verification based on the task (Landers and Doryab, 2023). However, the survey does not focus on testing RL in a conventional sense. It overlooks testing RL approaches, including mutation testing, differential testing, and other evaluation strategies necessary for practical RL system implementations.

A recent review by Nayab and Wotawa explored testing RL applications and employing RL for testing. Nayab and Wotawa (2024). However, the review focuses on RL applications in testing and lacks detailed insights into testing frameworks and approaches for RL. While current reviews on testing ML may provide a foundational understanding, they often neglect the specific challenges associated with testing RL systems. Reviews such as Marijan and Gotlieb (2020) emphasize the need for comprehensive testing to ensure the correctness and trustworthiness of ML systems but lack a focus on RL. Chen et al. address fairness testing in ML software without discussing fairness in RL contexts (Chen et al., 2024).

3. Research methodology

This review follows the systematic literature review process proposed in Kitchenham and Charters (2007). In the planning phase, a comprehensive review protocol was developed, encompassing six essential stages: defining research questions, designing a search strategy, selecting relevant studies, assessing their quality, extracting data, and synthesizing the findings. Each of these stages is detailed in the subsequent sub-sections.

3.1. Research questions

In this section, we propose seven research questions to guide our review. These questions focus on testing generation, objectives, methodologies, and emerging trends in the field, aiming to identify strengths and gaps in current testing practices while providing insights for improvement. The following sections are designed to provide thorough answers to the following research questions:

- **RQ1:** What are the main testing objectives in RL research, and how do the testing approaches vary based on these objectives?
- **RQ2:** Which components of RL systems are typically tested, and what testing methods (e.g., fuzzing, mutation, model-based, simulation-based) are used to assess key testing objectives like robustness, safety, and effectiveness?
- **RQ3:** How do agent types and environment complexity influence testing strategies and objectives?
- **RQ4:** What metrics evaluate test adequacy in testing RL systems?
- **RQ5:** What are the current research's main challenges and limitations in testing RL systems?
- **RQ6:** What research gaps exist in testing RL systems?
- **RQ7:** What are the potential future research directions for enhancing testing RL systems?

3.2. Search strategy

This section introduces the targeted scope, the methodology for paper collection, and the criteria for study selection. This survey covers software testing applied to RL systems, including all types.

3.2.1. Search terms

To ensure comprehensive yet focused coverage of relevant literature, we formulated the following search expression:

["Reinforcement Learning" AND ("Testing" OR "Verification" OR "Validation" OR "Model Checking")]

The search expression offers a well-balanced formulation that captures a broad but focused range of literature on RL testing. It includes core terms such as "Testing" to directly target work on test generation and execution, while "Verification" and "Model Checking" extend coverage to formal methods commonly used in safety-critical RL applications. The inclusion of "Validation" helps account for domain-specific terminology variations, especially in applied fields where "validation"

Table 3

Detailed search process per each digital library.

Digital library	Paper sections	Filter	Period	Publication type	Results
IEEE	Title, Abstract, Keywords	Reinforcement learning, software testing, and their relevance	2005–2024	Conference, Journal	2479
ACM	Title, Abstract, Keywords	–	–	–	76
Scopus	Title, Keywords	Computer science, engineering, software testing, final publication stage	–	Conference, Journal	3159
ArXiv	Title	–	–	–	134
ResearchGate	–	Since 2021, Full-Text-Only	Since 2021	Conference, Journal	112,084(10)
ScienceDirect	Title, Abstract, Keywords	SINCE 2015, CS, SE, Research Articles	–	–	2545

Table 4

Inclusion and exclusion criteria applied in paper selection process.

Phase	Exclusion	Inclusion	Description
1,2		✓	Reinforcement learning testing or relevance is explicitly mentioned in the paper's title, abstract, keywords
3		✓	Paper primarily focuses on testing RL
1	✓		Survey or review paper
3		✓	Study includes a clear description of the testing design and experiment
3		✓	Results of the testing experiment reported
3		✓	Work validates its findings through internal comparisons or external benchmarking
1,2	✓		Paper applies RL techniques to traditional or ML systems testing
1,2	✓		Paper targets RL problems without employing testing techniques
3	✓		Paper evaluates or enhances RL systems instead of testing it
1,2	✓		A paper that focuses on testing an application or system, where RL is a component rather than the primary focus of testing
1	✓		Works in progress paper
1	✓		Paper in non-English
3	✓		Redundant paper

may be used in place of “testing”. While broader terms such as “Robustness” or “Adversarial” were considered, they were excluded from the final query to avoid introducing a large volume of irrelevant results. Overall, this expression strikes an effective balance between comprehensiveness and precision, enabling the retrieval of relevant studies without overwhelming noise.

3.2.2. Literature resources

Six electronic databases were searched: IEEE Xplore, ScienceDirect (Elsevier), ArXiv, Scopus, ACM, and ResearchGate. The search query was tailored to meet the specific requirements of different digital libraries. The search covered titles, abstracts, and keywords for conference and journal articles. To further refine the search, we filtered the results to focus on computer science, reinforcement learning, and software testing. We modified Search coverage and filtering when available for better results. We retrieved 8259 studies (5650 after removing duplicate studies) from IEEE Xplore, ACM, Scopus, and ScienceDirect, as well as 134 studies from ArXiv, using the specified search term. 577 in ACM, 219 in Scopus, and 64 in ArXiv. We excluded ResearchGate due to the broad range of research results, even after applying stricter filters to research specifications, as the minimum number of results we obtained was 112,084 (10). Table 3 outlines the searching process and results per database.

3.2.3. Search selection process

Our exclusion underwent three phases: title screening, abstract screening, and full-text assessment. We applied the first and second screenings to studies from the ArXiv online database because there was no option to export the search results. We identified 23 studies after title screening and ended with 19 after abstract screening. Also, we exported 8259 studies from the other four databases and then integrated them into Mendeley Reference Manager. First, duplication was checked. After removing duplicates, we are left with 5650 unique studies. Title screening followed this, and we left with 30 studies. After the abstract screening, we excluded eight studies, leaving us with 22. We further removed five duplicates between ArXiv and the four databases. Therefore, the first and second exclusion phases yielded 36 papers from the five databases.

We applied forward and backward snowballing to identify additional relevant studies. In forward snowballing, we examined the citations of relevant studies to identify newer articles that referenced

them. In backward snowballing, we reviewed the reference lists of selected studies to capture foundational works, acknowledging that many relevant studies might not have used our search terms. Furthermore, we contextualized our findings by analyzing the publication sequences of authors to identify emerging trends and by reviewing the related works sections. This approach facilitated further research, leading to the discovery of 38 additional relevant studies. As part of the testing framework validation, TOYBOX — a suite of environments for experimental evaluation of DRL — is compared with ALE (Bellemare et al., 2013) in the TOYBOX paper (Tosch et al., 2019). This comparison highlighted the continued relevance of ALE and led to the inclusion of the 2013 study, which introduced ALE to the collected studies, resulting in 39 papers through snowballing. Seventy-five relevant papers were identified, 36 through direct search and 39 via snowballing.

3.2.4. Study selection

This section describes selecting studies for inclusion in the review based on the full-text assessment (third phase). We evaluated the quality of the 75 studies identified through the paper collection process, using predefined inclusion and exclusion criteria that focused on their contribution to testing RL and the clarity of the proposed approaches. Although a numerical scoring system was not employed, each study was evaluated based on whether it contributed to testing RL objectives and methodology. Given that testing RL is a relatively new field with limited literature, we adopted a more flexible approach to quality assessment. We aimed to identify weaknesses in the current research and provide suggestions for organizing future work in a way that builds knowledge in testing RL. Thus, our assessment considered less stringent criteria, allowing for an acknowledgment of all current efforts in the field. (Table 4 describes the inclusion and exclusion criteria applied in the three phases). Of the 75 studies evaluated in full text, 49 met the quality assessment criteria either entirely or partially. A study was excluded because it lacked experimental details and results. Six studies were excluded because their contributions were general (e.g., AI systems and Markov Decision Processes). Eight studies were excluded because their proposals focused on improving the agent's learning process or robustness rather than identifying potential issues or vulnerabilities in the system's behavior. Furthermore, six relevant articles were excluded due to redundancy, as they proposed model checking using formal verification, whereas we included similar papers. Five studies were excluded for focusing on domain-specific applications that integrated RL rather than testing RL components.

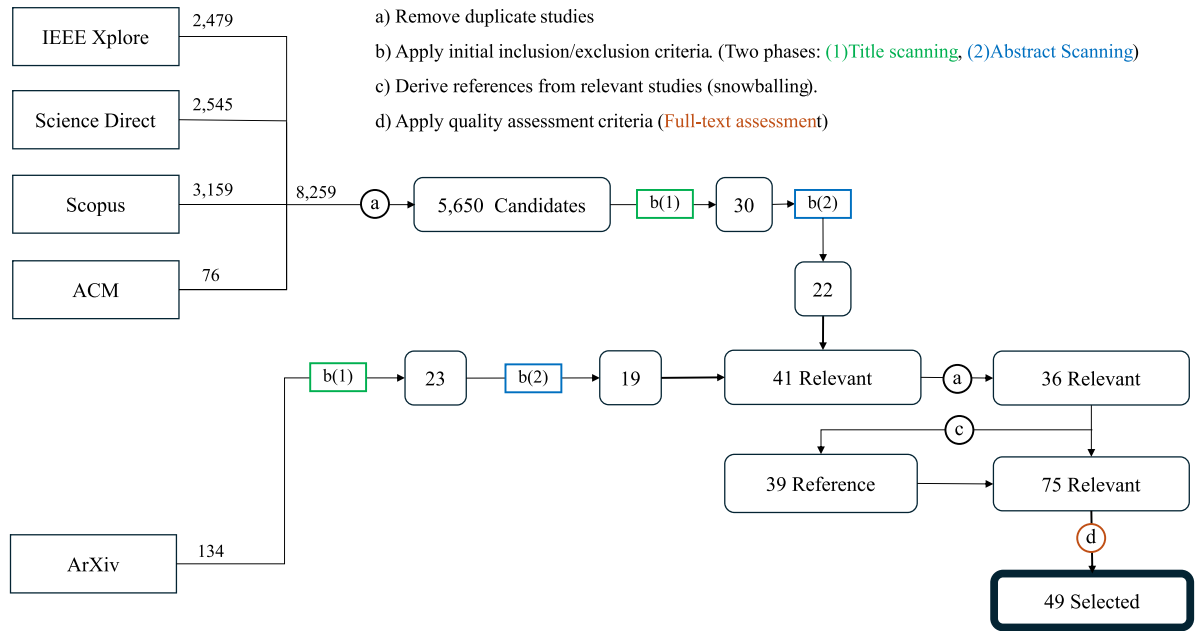


Fig. 3. Search and selection process.

Table 5

Data extraction form.

Item	Description
Study identifier	Unique ID for the study
Authors	Names of the authors
Year	Year of publication
Proposed framework	Description of the proposal and its objective
Study contribution	Unique Contributions study makes to RL testing as mentioned
Testing technique	Applied testing techniques (e.g., mutation testing, differential testing)
RL model	RL model was used as the “victim” in experiments
RL tested Objective	If mentioned, aspect of the RL model was specifically assessed (e.g., robustness, reliability)
Problem domain	The domain of the problem handled in the victim system
RL task	The specific environment or task utilized in the experiment
Targeted RL components	Specifying which components of the RL system (e.g., environment, agent, policy) were targeted by the proposed testing
Findings	The main findings as stated in the study
Validation	The validation approaches used (e.g., internal or external comparisons)
Limitation	Study limitations and threats to validity as mentioned
Gaps/Future work	Authors’ identified gaps or suggestions for future work

As a result, we ended with 49 studies focused solely on testing RL systems. A summary of the study collection and selection process is shown in Fig. 3.

3.2.5. Data extraction

Data was extracted from the selected studies using a standardized form to capture key details such as contributions, proposed approaches, results, and limitations, as shown in Table 5. Only explicitly stated information was considered, avoiding any inferences or assumptions. This structured format facilitates a clear comparison of methodologies across studies, providing insights into their applicability. This approach ensures a comprehensive understanding of testing RL practice.

4. Results and observations on RL testing research

This section presents a meta-analysis of existing literature on testing RL systems, aiming to quantitatively and qualitatively assess the development and current state of the field. Tables 6–9 present a comprehensive analysis of existing research on testing reinforcement learning (RL) systems. Each row corresponds to a specific publication, annotated across multiple dimensions including agent and environment types, problem domains, testing objectives, applied testing approaches, tested components, and validation strategies. This structured overview highlights the diversity of methods and focus areas across the literature,

helping to identify common trends as well as underexplored aspects in RL testing.

In the following subsections, we present detailed statistics and insights that reflect both the current landscape and the evolution of the RL testing field. We begin by analyzing the distribution of research focus across various testing aspects (Section 4.1), followed by a chronological overview of developments from 2013 to May 2025 (Section 4.2). We then examine publication patterns, including the proportion of peer-reviewed versus non-peer-reviewed works, the types of venues, and the distribution by publisher (Section 4.3). Finally, we review citation trends to highlight the increasing academic attention the field has received in recent years (Section 4.4).

4.1. Distribution of research focus across RL testing aspects

Fig. 4 illustrates the distribution of collected papers across various aspects of reinforcement learning (RL) testing. The most frequently addressed area is test input generation, with 36 papers focusing on this topic, highlighting its centrality in RL testing research. Testing objectives and test adequacy also receive considerable attention, reflecting growing interest in evaluating what and how RL systems are tested. In contrast, aspects such as test prioritization, debug and repair, fault taxonomy, and test oracle are much less explored, suggesting these

Table 6

A comprehensive analysis of testing RL work (Part 1).

Reference	Agent types						Environment types					Problem domain						Testing objectives												
	Deterministic	Stochastic	Model-Free	Off-Policy	On-Policy	Single-Agent	Multi-Agent	Deterministic	Stochastic	Partially-Observable	Fully-Observable	Simulated	Real-World	Health	Game	Corporate	Sport	Control	Autonomous	Robustness	Safety	Performance	Correctness	Stability	Resilience	Scalability	Reliability	Adaptability	Generalization	
Wan et al. (2024)		✓	✓	✓	✓				✓			✓			✓							✓						✓		✓
Zhang et al. (2024)			✓	✓	✓	✓			✓			✓			✓			✓				✓	✓					✓		
Tappler et al. (2022)		✓	✓	✓		✓			✓			✓			✓					✓	✓	✓				✓	✓			
Tambon et al. (2023)			✓	✓	✓	✓			✓			✓						✓												
Lu et al. (2022)			✓	✓		✓						✓						✓		✓		✓								
Tambon et al. (2023)			✓	✓	✓	✓			✓		✓	✓						✓										✓		
Mazouni et al. (2024)	✓		✓			✓		✓				✓						✓				✓								
Zolfagharian et al. (2023)	✓		✓	✓		✓			✓		✓	✓						✓			✓		✓							
Liu et al. (2024)			✓	✓	✓	✓						✓						✓		✓		✓								
Shi et al. (2024)			✓	✓	✓	✓						✓	✓					✓				✓								
Casper et al. (2023)			✓		✓		✓					✓	✓				✓			✓		✓								
Varshosaz et al. (2023)			✓	✓	✓	✓						✓	✓					✓						✓						
Tappler et al. (2024b)			✓	✓		✓					✓	✓			✓					✓	✓	✓								
Zhou and Liu (2023)		✓	✓	✓			✓		✓	✓		✓			✓					✓										
Guo et al. (2022)		✓	✓	✓	✓		✓		✓	✓		✓			✓					✓		✓								
Biagiola and Tonella (2024)		✓	✓			✓						✓							✓											
Biagiola and Tonella (2022)			✓	✓	✓	✓						✓						✓											✓	
Trujillo et al. (2020)			✓	✓		✓					✓	✓						✓				✓								
Zolfagharian et al. (2023)			✓	✓		✓			✓			✓						✓		✓										
Mazouni et al. (2025)			✓			✓		✓				✓						✓												
Li et al. (2022b)			✓						✓													✓						✓		
Eniser et al. (2022)	✓	✓	✓	✓	✓	✓		✓	✓	✓		✓						✓	✓				✓							
Ma et al. (2024)			✓	✓	✓		✓			✓		✓			✓	✓				✓										
Phan et al. (2020)		✓					✓		✓	✓		✓							✓	✓				✓	✓				✓	
Li et al. (2023)		✓				✓	✓		✓			✓				✓			✓	✓								✓		

areas remain underrepresented and present opportunities for further investigation.

4.2. Evolution of RL testing research

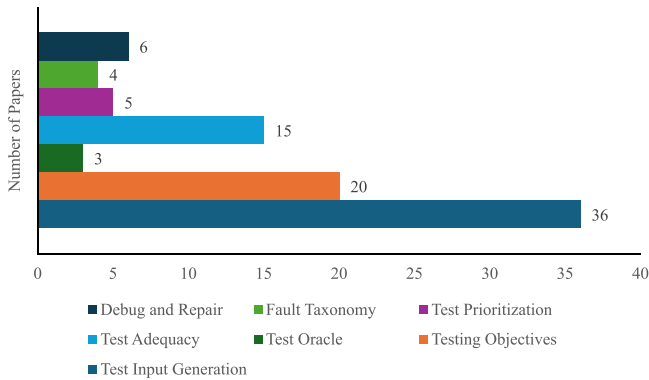
Testing RL is a recent and rapidly evolving field, as shown in Fig. 5. The concept of testing RL was introduced in Bellemare et al. (2013) with the Arcade Learning Environment (ALE), a platform for evaluating the general AI competency of RL agents across diverse Atari 2600 game environments. Following ALE, a significant gap existed in the field until December 2019, when Uesato et al. explored using historical data to develop adversarial testing methodologies (Uesato et al., 2019). In 2020, neuron coverage was investigated as a metric for testing RL adequacy (Trujillo et al., 2020), while antagonist agents were used for cooperative multi-agent reinforcement testing (Phan et al., 2020).

The field experienced significant growth in 2022; statistical testing was conducted for RL agent policies (Li et al., 2022a), and the first fault taxonomy for DRL programs, along with a model-based testing approach built on this taxonomy, was introduced (Nikanjam et al., 2022). Additionally, nine publications adapting conventional ML testing techniques to RL, including mutation (Lu et al., 2022), metamorphic (Eniser et al., 2022), fuzzing (Eniser et al., 2022), search-based (Tappler et al., 2022; Biagiola and Tonella, 2022), and multi-aspect adversarial attacks (Guo et al., 2022). This upward trend continued in 2023, with increased publications, including the differential evolution-based adversarial testing approach (Zhou and Liu, 2023). Publications in reinforcement learning testing notably peaked in 2023 and 2024, with twelve each year reflecting a growing interest in the field. At the beginning of 2025, a study implemented a transfer testing approach for agents in competitive environments to assess adversarial testing methods (Chen

Table 7

A comprehensive analysis of testing RL work (Part 2).

Reference	Testing approaches											Altered components						Validation					
	Mutation	Adversarial	Generative-based	Differential	Metamorphic	Gradient-based	Fault-Injection	Symbolic-Execution	Model-Based	Search-based	Formal-Verification	Fuzzing	Data-driven	Simulation-Based	Policy	Agent-learning	State	Action	Environment	Reward	State-of-Art	Human	Internal
Wan et al. (2024)		✓				✓						✓				✓					✓		✓
Zhang et al. (2024)							✓					✓								✓		✓	✓
Tappler et al. (2022)										✓		✓						✓			✓		✓
Tambon et al. (2023)	✓														✓		✓		✓	✓	✓		✓
Lu et al. (2022)	✓						✓								✓		✓		✓	✓	✓		✓
Tambon et al. (2023)	✓														✓		✓		✓	✓	✓		✓
Mazouni et al. (2024)										✓							✓		✓	✓	✓		✓
Zolfagharian et al. (2023)										✓											✓		✓
Liu et al. (2024)		✓															✓	✓					✓
Shi et al. (2024)															✓		✓	✓					✓
Casper et al. (2023)		✓													✓		✓				✓		✓
Varshosaz et al. (2023)	✓																						✓
Tappler et al. (2024b)										✓		✓					✓						✓
Zhou and Liu (2023)		✓				✓				✓							✓	✓	✓		✓		✓
Guo et al. (2022)		✓				✓											✓	✓		✓			✓
Biagiola and Tonella (2024)	✓									✓									✓		✓		✓
Biagiola and Tonella (2022)																			✓		✓		✓
Trujillo et al. (2020)																✓							✓
Zolfagharian et al. (2023)	✓									✓					✓						✓		✓
Mazouni et al. (2025)	✓											✓					✓				✓		✓
Li et al. (2022b)	✓	✓				✓						✓								✓			✓
Eniser et al. (2022)					✓							✓					✓						✓
Ma et al. (2024)										✓		✓					✓				✓		✓
Phan et al. (2020)		✓													✓		✓						✓
Li et al. (2023)			✓							✓							✓				✓		✓

**Fig. 4.** Focus areas of collected papers on testing aspects.

et al., 2025). By the end of February 2025, DRLMutation was published, a comprehensive framework for testing DRL systems that inject faults into the training process and the model of the trained agent (Li et al., 2025).

It is worth noting that ALE was introduced in 2013 as an RL benchmark environment primarily for validating Atari-based DRL. Its utilization increased with the rise of DRL work and techniques. However, with the introduction of OpenAI Gym in 2016, which offers flexibility across more tasks besides gaming, ALE's popularity declined. OpenAI Gym's simplicity and diverse environment support, including Atari and other benchmarks, have made it the preferred framework for testing RL. This trend is reflected in the declining use of ALE for testing RL studies since 2020 (See Fig. 6).

4.3. Peer review status and publication venues in RL testing research

Fig. 7 indicates that 88% of the reviewed papers (43 out of 49) were published in peer-reviewed venues, reflecting the academic rigor

of the field. Among these, 15 were made available on arXiv before their formal publication, and 9 of those preprints received 17 citations while still in the preprint stage. The corresponding formal versions appeared in reputable venues such as IEEE, ACM, Springer, and the AI Access Foundation. However, no venue consistently attracts research on testing RL. This dispersion may be due to uncertainty about the disciplinary identity of testing RL, whether rooted in computer science, artificial intelligence, or software engineering. This is evident in the wide range of publication venues, including conferences and journals spanning software testing, AI, machine learning, and systems engineering, as shown in Fig. 8.

As indicated in Fig. 9(a), 68% of the reviewed papers were published in conference proceedings, reflecting the field's fast pace and preference for rapid dissemination of results. Journals account for 29% of the publications, while 13% remain unpublished or available as preprints. Fig. 9(b) shows that IEEE and ACM are the most prominent publishers, accounting for 22% and 20% of the reviewed papers, respectively. This may be due to their strong presence in both software engineering and AI communities and their openness to emerging interdisciplinary topics, such as RL testing.

4.4. Citation evolution

Fig. 10 illustrates a clear upward trend in the number of citations received by the reviewed papers, as reported by Scopus, increasing from a single citation in 2019 to 140 citations in 2024, with a cumulative total of 277 by early 2025. The consistent growth in citations underscores the increasing recognition of testing RL as a crucial aspect of ensuring the reliability and safety of RL and RL-integrated systems.

5. Research questions: Findings and discussion

In the following subsections, we will address the research questions outlined in Section 3.1. Each RQ will be explored in its dedicated section.

Table 8

A comprehensive analysis of testing RL work (Part 3).

Reference	Agent types						Environment types					Problem domain					Testing objectives												
	Deterministic	Stochastic	Model-Free	Off-Policy	On-Policy	Single-Agent	Multi-Agent	Deterministic	Stochastic	Partially-Observable	Fully-Observable	Simulated	Real-World	Health	Game	Corporate	Sport	Control	Autonomous	Robustness	Safety	Performance	Correctness	Stability	Resilience	Scalability	Reliability	Adaptability	Generalization
He et al. (2024)						✓	✓					✓				✓			✓		✓								
Nikanjam et al. (2022)			✓	✓		✓		✓				✓			✓			✓					✓						
Thomas et al. (2024)		✓	✓	✓	✓	✓		✓	✓		✓	✓						✓		✓									
Tappler and Aichernig (2023)		✓	✓	✓		✓			✓	✓		✓			✓						✓	✓							
Uesato et al. (2019)	✓			✓	✓							✓						✓	✓		✓						✓		
Chen et al. (2025)							✓	✓			✓	✓						✓	✓										
Tappler et al. (2024a)			✓	✓	✓	✓		✓	✓		✓	✓						✓										✓	
Morán et al. (2023)		✓	✓		✓	✓			✓		✓	✓																	
Li et al. (2022a)		✓	✓	✓					✓				✓	✓															
Allen et al. (2021)	✓	✓	✓	✓		✓		✓	✓	✓		✓		✓				✓	✓			✓							
Mathieu et al. (2023)	✓	✓	✓	✓	✓	✓		✓				✓						✓				✓							
Behzadan and Hsu (2019)			✓	✓	✓							✓																	
Bacci and Parker (2020)												✓						✓		✓									
Mqirmi et al. (2021)			✓	✓								✓						✓			✓								
Zolfagharian et al. (2024)	✓		✓	✓		✓						✓				✓		✓			✓								
Raz et al. (2022)	✓	✓										✓								✓		✓							
Bellemare et al. (2013)			✓		✓	✓						✓			✓							✓							✓
Tosch et al. (2019)	✓	✓	✓	✓	✓	✓		✓	✓			✓			✓					✓		✓							✓
Tappler and Lorber (2024)	✓			✓		✓			✓		✓	✓			✓						✓								
Li et al. (2025)						✓			✓		✓	✓						✓	✓	✓			✓				✓		
Corsi et al. (2020)				✓		✓			✓		✓	✓						✓	✓		✓	✓				✓	✓		
Gross and Spieker (2024)		✓			✓	✓			✓		✓	✓			✓			✓			✓								
Zhi et al. (2024)	✓		✓	✓		✓		✓			✓	✓						✓		✓		✓							
Pranger et al. (2024)	✓					✓		✓			✓	✓			✓			✓	✓		✓								

5.1. RQ1. What are the main testing objectives in RL research, and how do the testing approaches vary based on these objectives?

According to our overview in Section 2.2.9 and the analysis in Tables 6–9, testing RL has mainly focused on robustness, resilience, and safety. Seven studies focused on reliability, robustness, or resilience (Phan et al., 2020; Guo et al., 2022; Lu et al., 2022; Liu et al., 2024; Zhou and Liu, 2023; Uesato et al., 2019; Zhi et al., 2024), nine examined safety (Tappler and Aichernig, 2023; Tappler et al., 2024b, 2022; Bacci and Parker, 2020; Mqirmi et al., 2021; Zolfagharian et al., 2024; Corsi et al., 2020; Gross and Spieker, 2024), one assessed correctness (Varshosaz et al., 2023), and one evaluated adaptability (Biagiola and Tonella, 2022). Robustness and resilience are tested by stressing agents under adversarial scenarios. Performance is assessed under disturbances, while safety is measured by the ability to avoid unsafe states.

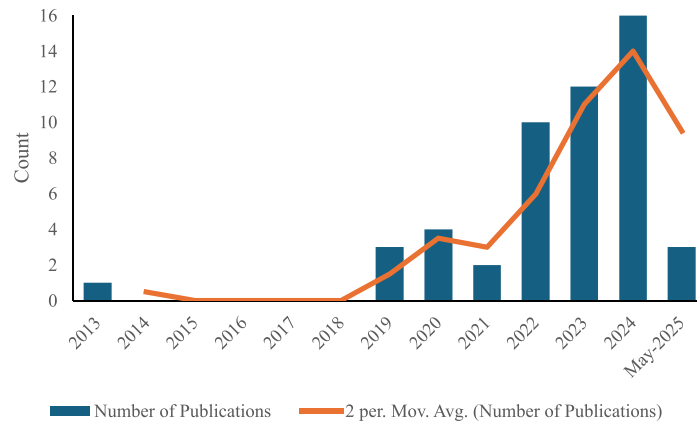
Correctness is assessed based on formal specifications, and adaptability is evaluated by how agents adjust to diverse environments.

RL safety testing methods differ in their strategies, underlying assumptions, and applicability across agent types. Uesato et al. utilize a learned failure probability model to guide the generation of adversarial inputs (Uesato et al., 2019). Tappler et al. focus on fuzzing boundary states and iteratively improving policies through learned repairs (Tappler et al., 2024a,b). Tappler and Aichernig advance differential testing, highlighting that high-performing agents (by reward) may still be unsafe (Tappler and Aichernig, 2023). Zolfagharian et al. propose execution-based failure detection or predictive runtime monitoring; however, this approach is limited to discrete Q-learning agents (Zolfagharian et al., 2023, 2024). Mqirmi et al. and Bacci and Parker apply formal verification: the former targets MARL with model checking, while the latter offers probabilistic guarantees through abstract interpretation, requiring deterministic environments (Mqirmi et al., 2021;

Table 9

A comprehensive analysis of testing RL work (Part 4).

Reference	Testing approaches												Altered components						Validation				
	Mutation	Adversarial	Generative-based	Differential	Metamorphic	Gradient-based	Fault-Injection	Symbolic-Execution	Model-Based	Search-based	Formal-Verification	Fuzzing	Data-driven	Simulation-Based	Policy	Agent-learning	State	Action	Environment	Reward	State-of-Art	Human	Internal
He et al. (2024)										✓							✓				✓		✓
Nikanjam et al. (2022)									✓	✓		✓											✓
Thomas et al. (2024)	✓														✓	✓	✓		✓	✓	✓		✓
Tappler and Aichernig (2023)				✓					✓														✓
Uesato et al. (2019)		✓																					✓
Chen et al. (2025)		✓																✓					✓
Tappler et al. (2024a)				✓					✓														✓
Morán et al. (2023)	✓						✓										✓		✓	✓			✓
Li et al. (2022a)																							✓
Allen et al. (2021)														✓					✓				✓
Mathieu et al. (2023)																							✓
Behzadan and Hsu (2019)		✓																					
Bacci and Parker (2020)																	✓						✓
Mqirmi et al. (2021)												✓											✓
Zolfagharian et al. (2024)																							✓
Raz et al. (2022)							✓										✓	✓				✓	✓
Bellemare et al. (2013)																						✓	✓
Tosch et al. (2019)																						✓	✓
Tappler and Lorber (2024)				✓						✓					✓		✓	✓					✓
Li et al. (2025)	✓						✓								✓		✓	✓		✓	✓		✓
Corsi et al. (2020)											✓				✓		✓	✓				✓	✓
Gross and Spieker (2024)											✓				✓		✓	✓				✓	✓
Zhi et al. (2024)											✓				✓		✓			✓	✓		✓
Pranger et al. (2024)											✓				✓		✓				✓		✓

**Fig. 5.** Number of publications in testing RL systems (from 2013 to May 2025).

Bacci and Parker, 2020). These works demonstrate diverse yet complementary techniques for safety testing in RL, each with trade-offs in terms of generality, interpretability, and data or model assumptions.

Methods often target robustness or performance degradation in RL systems under adversarial or diverse environmental conditions. While black-box fuzzing techniques like MDPFuzz, AgentFuzz, and CureFuzz dynamically mutate inputs or states to uncover faults, their effectiveness varies based on test scope (e.g., initial vs. full trajectory mutations) (Mazouni et al., 2025; Pang et al., 2022; Li et al., 2022b; He et al., 2024). Model-guided approaches, such as Biagiola's surrogate-based search, leverage learned classifiers to efficiently predict failure-inducing scenarios, offering higher efficiency in failure detection (Biagiola and Tonella, 2024). Some methods (e.g., Zhou and Liu, 2023) rely on access to policy internals, which limits their generalizability, while others, such as CureFuzz (Li et al., 2022b), maintain black-box applicability through novelty-driven exploration. These works demonstrate that traditional robustness metrics (e.g., cumulative

reward) can mask policy fragility under edge cases, underscoring the need for more comprehensive test metrics and scenario diversity. Table 10 summarizes RL Safety and Robustness Testing Methods.

Testing methods vary by objective. Robustness is typically evaluated through mutation scores, reward variations under adversarial conditions, and performance degradation across environments. Correctness assessment remains challenging in stochastic environments due to the exploration–exploitation trade-off, though it is more tractable in deterministic settings. Adaptability testing modifies environmental parameters and monitors the agent's ability to adapt, often measuring regression or adaptation probabilities. Safety testing examines the behavior of an agent in near-critical or unsafe states to detect potential violations. However, there is currently no unified or standardized setting for testing RL systems—each study employs different metrics, tasks, and RL algorithms. As a result, direct and fair comparison between testing outcomes is largely infeasible. Instead, we focused our synthesis on comparing the methodologies and the technical improvements introduced by each approach.

Table 10
Comparative summary of RL safety and robustness testing methods.

Reference	Technique	Test target	Required inputs/data	Output metric	Applicable systems	Limitation (s)	Key insight/conclusion
Uesato et al. (2019)	Adversarial testing + AVF	Failure search via initial states	Learned failure probability predictor, training data	Failure probability estimate	General DRL	Assumes similar failure modes across agents	Predictive models improve search for rare failures
Tappler et al. (2024a,b)	Search-based fuzzing + Repair	Boundary states, policy repair	Reference traces, fuzzed traces	Safety coverage, pass/fail of test suites	General DRL (black-box)	Dependent on trace access and fuzzing coverage	Fuzzing aids both evaluation and repair of unsafe policies
Tappler and Aichernig (2023)	Differential testing	Policy comparison via failure rates	Manually defined abstraction, initial state clusters	Failure frequency difference	Deep RL (Super Mario Bros.)	Requires domain knowledge for abstraction	Reward is insufficient to assess safety; differential testing reveals discrepancies
Zolfagharian et al. (2023)	Genetic algorithm + ML-guided testing	Functional faults in episodes	Executed episodes, simulations, ML classifiers	Faulty episode detection	DRL systems (CartPole, etc.)	Ignores reward faults; simulation is costly	Search-based testing reveals hard-to-find execution faults
Zolfagharian et al. (2024)	Black-box runtime monitoring	Safety violations during execution	Q-values, agent behavior, state abstraction	Early safety violation prediction	Q-learning with discrete deterministic policies	Restricted to discrete Q-learning	Enables early prediction of violations with low false positives
Mqirmi et al. (2021)	Formal verification (PCTL)	Multi-agent policy compliance	Abstract model, PCTL specs	Formal property satisfaction	Multi-agent RL (cooperative)	Complex abstraction; cooperative only	Model checking ensures safety compliance in MARL
Bacci and Parker (2020)	Probabilistic model checking + abstraction	DRL controller behavior under uncertainty	Full deterministic environment model, NN controller	Prob. of reaching failure within horizon	DRL in known deterministic settings	Inapplicable to unknown/uncertain environments	Offers probabilistic safety guarantees from abstracted dynamics
Guo et al. (2022)	Adversarial testing	c-MARL behavior	State/action/reward access	Performance degradation	c-MARL	Assumes agent access; limited to known failure modes	Shows multi-space attack impact
Zhou and Liu (2023)	DE + FGSM	Team policy	Policy network, env state	Team reward	c-MARL, discrete only	Requires full policy/env access	Shows reward drop $\neq$ safety violation
Chen et al. (2025)	Transfer + perturbation	Critical states	Source task data	Reward, failure coverage	DRL	Needs similar source task	Expands failure space via imitation
Pang et al. (2022) and Li et al. (2022b)	Fuzzing (initial state)	Agent + Env	Black-box access	Failure rate, coverage (GMM)	DRL	Misses mid-run faults; heavy coverage calc	Finds robustness gaps with minimal assumptions + Simpler fuzzer may outperform MDPFuzz
Li et al. (2022b)	Fuzzing loop	System behavior	State mutation, performance	Reward drops	DRL	Limited evaluation; early stage	Promising system-level fuzzing
Biagiola and Tonella (2024)	Surrogate + Search	Env config	Training failure data	Failures, diversity	DRL	No mid-execution faults; dependent on model accuracy	Effective and efficient fault finder
He et al. (2024)	Curiosity-based fuzzing	Full system	Curiosity-based state novelty	Crash count, state coverage	SDM (incl. DRL)	No internal insight; black-box only	Finds novel diverse crash scenarios
Tappler and Lorber (2024)	Model-based test generation via abstract MDP + coverage analysis	Policy, State, Action	Trained DQN agent, environment traces, learned MDP	Failure rate, Neuron Coverage, State importance	Planning-based DRL	Limited to interpretable states, NC not reliable, not generalizable to latent-state models	Branching-point test selection improves failure detection; NC is inadequate for test adequacy in DRL

(continued on next page)

Table 10 (continued).

Reference	Technique	Test target	Required inputs/data	Output metric	Applicable systems	Limitation (s)	Key insight/conclusion
Li et al. (2025)	Mutation testing (107 DRL-specific operators)	Training process and model components	Trained DRL agents, environments	Mutation score, failure rate, reward drop	Model-free DRL agents	No formal oracle; not evaluated in real-world settings; assumes internal access	Mutations reveal fault-sensitive components and assess test suite effectiveness across DRL pipelines
Corsi et al. (2020)	Semi-formal verification using interval analysis	Policy output (action preferences)	Trained DRL policy, safety property, input intervals	Safe rate, Violation rate	Model-free DRL agents	no real-time integration; scalar-output comparison only	SFV efficiently identifies unsafe behaviors missed by reward-based evaluation; scalable alternative to formal methods.
Gross and Spieker (2024)	Probabilistic model checking using PCTL over DTMCs	Trained stochastic policy	DRL policy, environment model, safety property (PCTL)	Probability of property satisfaction	Model-free, memoryless DRL agents in discrete environments	Requires discrete states/actions; supports memoryless policies only	Accurately detects safety violations in stochastic agents; scalable alternative to deterministic verification
Zhi et al. (2024)	Formal verification using reward martingales	Policy behavior under perturbed inputs	Trained DRL policy, state perturbation model	Upper/lower reward bounds, tail probability	Deterministic, model-free DRL	High computational cost in high dimensions; supports deterministic policies only	Reward martingales enable post-training robustness certification through bounded reward guarantees

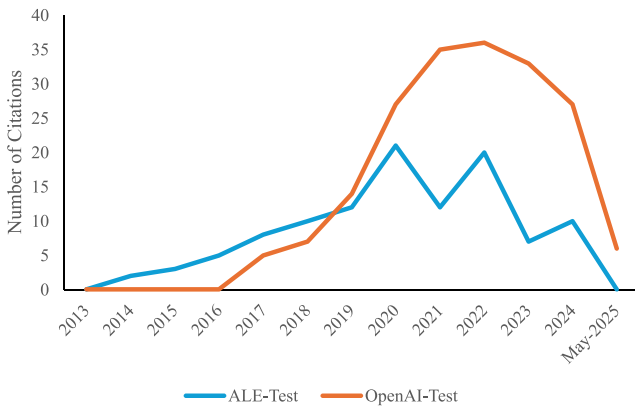


Fig. 6. Trend of validation and testing using ALE and OpenAI (2013-May 2025).

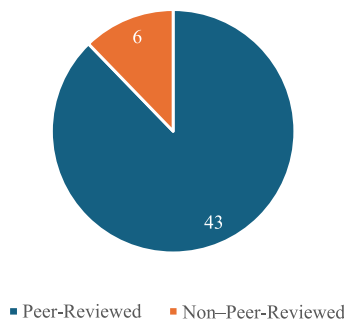


Fig. 7. Distribution of peer-reviewed vs. non-peer-reviewed Studies.

5.2. RQ2. Which components of RL systems are typically tested, and what testing methods (e.g., fuzzing, mutation, model-based, simulation-based) are used to assess key testing objectives like robustness, safety, and effectiveness?

Regarding test generation, nine papers applied adversarial testing to assess vulnerabilities in RL systems. For example, Uesato et al. proposed

a method for evaluating failure risks by estimating the probability of catastrophic failures using adversarial testing, Casper et al. generated adaptive adversarial policies using internal state information, Phan et al. developed the Antagonist-Ratio Training Scheme for multi-agent systems, Guo et al. applied attacks to mislead cooperative agents, and Zhou and Liu utilized differential evolution to target critical agents (Uesato et al., 2019; Casper et al., 2023; Phan et al., 2020; Guo et al., 2022; Zhou and Liu, 2023). One paper developed a simulation environment for testing DRL agents solving the Ambulance Location Problem (Allen et al., 2021).

Seven papers applied fuzzing techniques to states, actions, traces, and rewards. For instance, π -fuzz generates diverse test cases using random environmental walks, while Mazouni et al. investigated MDPFuzz for RL fault discovery (Eniser et al., 2022; Mazouni et al., 2025; Pang et al., 2022). Additionally, Wan et al. combined search-based methods with gradient ascent-based mutations to create failure-inducing states (Wan et al., 2024). MASTest employed a state criticality table to prioritize test cases (Ma et al., 2024) seven papers focused on mutation testing; Lu et al. introduced controlled mutations at both the environment and agent levels, Tambon et al. defined eleven mutation operators, Thomas et al. generated diverse test configurations with a random test generator, and Li et al. proposed mutation testing framework tailored for DRL systems (Lu et al., 2022; Tambon et al., 2023; Thomas et al., 2024; Li et al., 2025). CureFuzz optimized the testing workflow by ranking seed tests based on their potential to trigger crashes and explore new states (He et al., 2024). Moreover, Varshosaz et al. proposed specification-based testing for standard RL methods, such as Q-learning, targeting agent-observation mappings, and termination conditions. Li et al. presented a generative-based testing framework that utilizes a diffusion model (Varshosaz et al., 2023; Li et al., 2023). Bacci and Parker applied model checking of MDPs, and Mqirmi et al. utilized formal verification to ensure safety constraints in multi-agent RL systems, representing formal method approaches (Bacci and Parker, 2020; Mqirmi et al., 2021). Nikanjam et al. introduced the DRLinter tool for analyzing structural integrity, Tappler and Aichernig combined automata learning with differential testing to assess the safety of RL agents, and Moran et al. focused on fault localization by analyzing episode outcomes to identify states linked to poor performance (Nikanjam et al., 2022; Tappler and Aichernig, 2023; Morán et al., 2023). Finally, Li et al. and Mathieu et al. applied

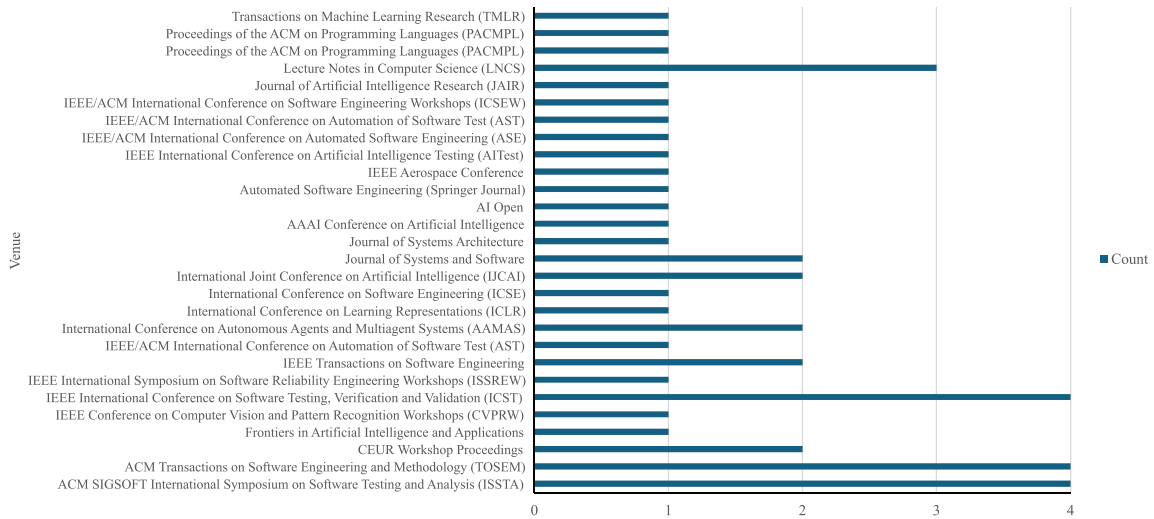


Fig. 8. Publication venues of included studies.

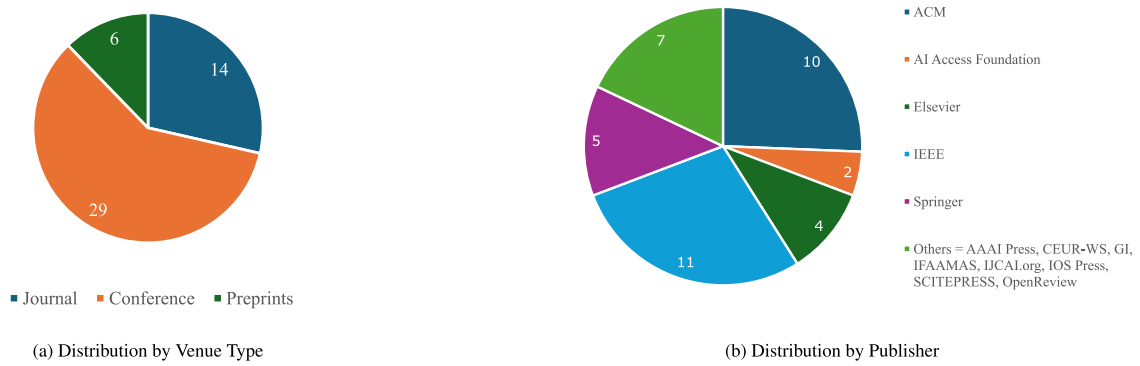


Fig. 9. Dissemination of RL testing publications by venue type and publisher.

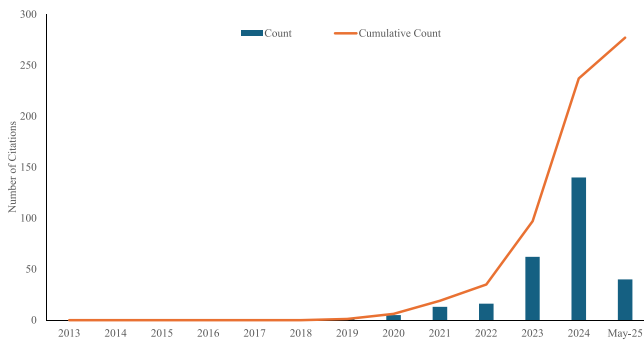


Fig. 10. Number of citations per year.

data-driven statistical testing to test the agent policy (Li et al., 2022a; Mathieu et al., 2023).

Adversarial testing effectively uncovers vulnerabilities by simulating challenging scenarios, while fuzzing explores diverse states to reveal potential bugs. Mutation testing assesses robustness by introducing controlled changes, and specification-based methods ensure adherence to defined properties. Formal methods provide rigorous guarantees by verifying system behavior against mathematical models. Runtime monitoring identifies safety violations during operation, offering real-time insights into agent behavior. Each approach varies

in effectiveness. Adversarial and mutation testing often yield more insights into system weaknesses, while fuzzing and generative techniques enhance state diversity in testing scenarios. Moreover, adaptive statistical tests offer computational efficiency by minimizing the number of runs needed for statistically significant conclusions in simpler systems. Simulation testing, tailored for specific tasks, simulates challenges but does not guarantee real-life performance in deployment.

CureFuzz is best for exploratory testing without prior failures, as it uses curiosity-driven exploration to uncover diverse scenarios (Li et al., 2022b). Indago excels in environments where failure demonstrations are available, leveraging surrogate models for efficient fault discovery (Biagiola and Tonella, 2024). MDPFuzz performs well in deterministic tasks like LunarLander, where gradient-based reward mutations are effective (Mazouni et al., 2025). DRLFuzz serves as a basic baseline but lacks adaptive or learning-based strategies (Wan et al., 2024). DRLMutation is most effective for comprehensive test suite evaluation, offering a broad set of mutation operators targeting both training and inference components (Li et al., 2025). RLMutation is better suited for dynamic adequacy assessment, evaluating runtime behavioral deviations between original and mutated agents in live settings (Tambon et al., 2023). CASTLE is ideal for model-based testing where understanding and manipulating agent behavior through learned MDPs is essential (Tappler et al., 2024a). AdaStop is suited for statistically sound performance comparisons between RL agents (Mathieu et al., 2023). Stationarity testing is valuable in offline or long-horizon learning scenarios to detect shifts or instabilities in agent behavior over time (Li et al., 2022a).

Components of RL systems included in agent testing are decision-making policies, learning algorithms, and internal state representations. Modifications typically applied during testing involve perturbations to policy parameters, alterations to reward structures, and adjustments to state feedback.

5.3. RQ3. How do agent types and environment complexity influence testing strategies and objectives

We classified the collected papers according to various categories: single-agent or multi-agent systems, model-based or model-free agents, on-policy or off-policy learning agents, deterministic or stochastic systems, continuous or discrete action spaces, and value-based, Q-learning, or Deep Q-learning agents. Not all papers provided complete information, with some mentioning all attributes, others partially. Despite this, the classification highlights trends and practices in selecting testing strategies and validation methods for research on RL systems testing.

Sixty-four percent of the papers focus on testing single-agent systems, 9.5% address multi-agent systems, and 9.5% evaluate their approaches on single-agent and multi-agent systems. Among multi-agent systems, the majority concentrate on cooperative systems, with only 30% focus on competitive ones.

The analysis shows that the environments used in targeted systems to evaluate testing approaches reveal a diverse range of settings, totaling forty-two distinct environments (see Table 11). There is a heavy reliance on simulations, with several classified as toy environments, such as Flappy Bird and the cliff-walking game. A significant portion focuses on DRL systems tested in medium-complexity simulated environments, such as the Bipedal Walker and Mountain Car, highlighting the prevalent use of simulations for benchmarking. Additionally, six papers evaluate RL systems within simple and discrete state environments, including car-stopping and grid-world tasks. The OpenAI Gym environments, such as Mountain Car, Cart Pole, Lunar Lander, and Bipedal Walker, are among the most frequently used for evaluating proposed testing approaches. However, there is limited representation of high-complexity environments. Multi-agent systems are tested in environments like CARLA for autonomous driving and Cooperative Navigation, indicating an engagement with complex, collaborative scenarios. Despite this breadth, a notable absence remains in testing RL systems in real-world RL applications.

Of the 49 collected papers, 42 proposed a testing approach for RL systems, with many providing details about the targeted RL systems. Among these, 36 focus on model-free systems. Specifically, 15 address off-policy and on-policy algorithms, 14 focus solely on off-policy methods, and two test on-policy algorithms. Regarding value-based and policy-based methods, 14 evaluate value-based systems, three focus only on policy-based methods, and eight address both. Additionally, seven papers test deterministic agents, ten test stochastic agents, and three evaluate both. Notably, algorithms such as PPO, DQN, and Q-learning are frequently mentioned in these studies. Most papers confirm their focus on testing DRL systems. See Table 12 for the distribution of research papers among tested RL algorithms.

The nature of RL agents, whether deterministic or stochastic, significantly shapes testing strategies. Deterministic agents' predictable behavior enables targeted tests for specific scenarios and outcomes, while stochastic agents' variability demands broader test coverage to capture diverse behaviors. Similarly, the dimensionality and complexity of state and action spaces influence testing approaches, requiring tailored methods to ensure comprehensive evaluation across varying conditions and interactions within the RL environment.

Existing research on testing RL systems focuses on simulated environments, especially in gaming, autonomous driving, and general control tasks. However, domains like healthcare, military applications, and finance are underexplored. Only one of the forty-two analyzed papers experimented with real-world data, specifically from the 2018 Intern Health Study (Li et al., 2022a).

5.4. RQ4. What metrics evaluate test adequacy in testing RL systems?

Twelve papers focus on testing adequacy approaches. Trujillo et al. and Tappler et al. assessed neuron coverage, commonly used in DNN testing (Trujillo et al., 2020; Tappler and Lorber, 2024). Tappler et al. achieved action and state coverage through a depth-first search. Tappler and Aichernig introduced observation coverage to evaluate if RL policies reach specified abstract states, including safety-critical scenarios (Tappler et al., 2022; Tappler and Aichernig, 2023). Li et al. applied grid-based clustering to track unique clusters and assess state diversity coverage (Li et al., 2023). Lu et al. and Varshosaz et al. measured adequacy using the mutation score, quantifying the proportion of detected mutants (Lu et al., 2022; Varshosaz et al., 2023). Mazouni et al. introduced fault diversity adequacy, assessing the variety of faults identified during testing (Mazouni et al., 2024). Uesato et al. proposed a failure probability-based coverage approach, estimating catastrophic failure likelihoods based on initial conditions (Uesato et al., 2019). Shi et al. applied for multi-granularity coverage, measuring state-level, action-level, and policy-level (state-action) coverage (Shi et al., 2024). He et al. implemented multi-objective test coverage, balancing exploration, and crash identification (He et al., 2024). Tappler et al. utilized cluster coverage (Tappler et al., 2024a). See Table 13 for a comparative summary.

5.5. RQ5. What are the current research's main challenges and limitations in testing RL systems?

Despite increasing attention to testing reinforcement learning systems, several open challenges and limitations hinder the development of robust, scalable, and generalizable testing methodologies. The following analysis categorizes and discusses these challenges based on recurring themes observed across the reviewed literature.

1. **General Limitations in Testing RL Systems:** Testing RL systems faces general limitations. Many approaches lack generality, restricting their use to specific tasks (e.g., Lu et al., 2022). Randomness in RL environments results in inconsistent outcomes, complicating the reliability of test outcomes (e.g., Lu et al., 2022; Biagiola and Tonella, 2024). Real-world applicability is limited, as most frameworks are tested only on benchmarking tasks (e.g., Mazouni et al., 2024; Casper et al., 2023; Allen et al., 2021). Additionally, many methods struggle to adapt to dynamic environments (e.g., Guo et al., 2022; Mazouni et al., 2024). Stochasticity in RL complicates consistent fault detection due to variations in agent performance. Mitigating this often involves running agents multiple times under identical conditions and averaging the result (Lu et al., 2022). However, this is resource-intensive. State-based testing complexity is compounded by the need for robust methods to address it, such as search-based optimization (Zolfagharian et al., 2023) and enhanced action-state coverage (Tappler et al., 2022). Efficiency remains a critical concern, as achieving sufficient coverage often requires high computational costs. For instance, white-box adversarial testing is practical during development but is limited across diverse environments (Casper et al., 2023). Test prioritization and reduction strategies can improve efficiency by focusing on critical scenarios. Fault identification is challenging in RL due to the difficulty of defining "bugs". Multiple runs with varying random seeds improve detection but require carefully designed reference policies (Eniser et al., 2022). Additionally, the probabilistic nature of RL introduces statistical flakiness, which can be mitigated by setting thresholds and conducting multiple tests (Varshosaz et al., 2023). Addressing these challenges requires diverse methodologies, efficient testing practices, and ongoing research to enhance the effectiveness and efficiency of testing.

Table 11

Overview of testing environments: Types and characteristics.

Task	Domain	Observability	Determinism	Complexity	Papers#
Flappy Bird (Talendar, 2021)	Gaming	Full	Stochastic	Low	1
Catcher	Gaming	Full	Stochastic	Low	1
Pong (OpenAI Gym, 2022g)	Gaming	Full	Stochastic	Low	1
Frozen Lake (OpenAI Gym, 2022c)	Benchmark	Partial	Stochastic	Medium	3
Mountain Car (OpenAI Gym, 2022f)	Benchmark	Full	Stochastic	Medium	8
Cart Pole (OpenAI Gym, 2022b)	Benchmark	Full	Deterministic	Medium	12
Lunar Lander (OpenAI Gym, 2022e)	Benchmark	Full	Deterministic	Medium	7
Super Mario Bros (Team, 2020)	Gaming	Full	Stochastic	Medium	3
Ant (OpenAI Gym, 2022a)	Benchmark	Partial	Stochastic	High	2
Half Cheetah (OpenAI Gym, 2022d)	Benchmark	Partial	Stochastic	High	2
Swimmer (OpenAI Gym, 2022h)	Benchmark	Partial	Stochastic	High	1
Walker (OpenAI Gym, 2022i)	Benchmark	Partial	Stochastic	High	2
Google Research Football (Kurach et al., 2020)	Gaming	Full	Stochastic	High	1
car stopping	General	Full	Deterministic	Low	1
grid-worlds	General	Full	Deterministic	Low	1
cliff walking	General	Full	Deterministic	Low	1
StarCraft II Challenge (Samvelyan et al., 2019)	Gaming	Partial	Stochastic	High	2
Coop Navi (Lowe et al., 2017)	Robotic	Partial	Stochastic	Medium	3
Bipedal Walker (Gym, 2022b)	Robotic	Partial	Stochastic	Medium	4
ACAS-Xu (Marston and Baca, 2015)	Aviation	Partial	Stochastic	High	2
CARLA (Dosovitskiy et al., 2017)	Driving	Partial	Stochastic	High	4
TORCS3D Car Racing (GitHub, 2016)	Driving	Partial	Stochastic	Medium	1
Humanoid Stand Gym (2022c)	Robotic	Partial	Stochastic	Medium	2
Highway (Leurent, 2018)	Driving	Partial	Stochastic	Medium	1
Taxi (Gym, 2022e)	Robotic	Full	Stochastic	Medium	3
Pendulum (Gym, 2022d)	Robotic	Full	Deterministic	Low	2
Acrobot (Gym, 2022a)	Robotic	Full	Stochastic	Medium	1
Walking Humanoid (Foundation, 2022)	Robotic	Partial	Stochastic	High	1
Parking Task	Robotic	Partial	Stochastic	Medium	1
Pumping Task	General	Full	Stochastic	High	1
Cyber-Physical Production System	Robotic	Partial	Stochastic	Medium	1
Minigrid Experiment	Gaming	Partial	Stochastic	Medium	1

This table provides a summary of various testing environments, categorized by their task domain, observability (Fully Observable or Partially Observable), determinism (Deterministic or Stochastic), and complexity (Low, Medium, or High). These attributes are generalized but may vary based on specific experimental settings.

Legend: Observability: Full = Fully Observable, Partial = Partially Observable; Determinism: Deterministic = Fixed outcomes, Stochastic = Variable outcomes; Complexity: Low = Simple dynamics, Medium = Moderate dynamics, High = Complex dynamics.

Table 12

Distribution of research papers among tested RL algorithms.

Algorithm	Type	Agent	Policy	Learning	Paradigm	Num of papers
DQNs	DRL	SA	VB	OP-OFP	Model-Free	23
A2C	DRL	SA	VB-PB	OP-OFP		5
PPO	DRL	SA	PB	OP		11
Q-Learning	RL	SA	VB	OFP		7
DDPG	DRL	SA	VB-PB	OFP		3
TRPO	DRL	SA	PB	OP		2
TD3	DRL	SA	VB	OFP		2
SAC	DRL	SA	VB-PB	OFP		5
Dueling DQN	DRL	SA	VB	OFP		2
D3QN	DRL	SA	VB	OFP		1
SARSA	DRL	SA	VB	OP		1
DQfD	DRL	SA	VB	OFP		1
QMIX	DRL	MA	VB	OFP		3
VDN	DRL	MA	VB	OFP		1
MAPPO	DRL	MA	PB	OP		1
MADDPG	DRL	MA	PB	OFP		1
Behavioral Cloning	RL	MA	VB-PB	OFP		1
IRL	RL	MA	VB-PB	OFP		1
TQC	DRL	SA	VB	OFP		1
DDQN	DRL	SA	VB	OFP		2
D4PG	DRL	SA	PB	OFP		2
Actor-Critic	DRL	SA	VB-PB	OP-OFP		2

Legend: SA — Single Agent, MA — Multi-agent, OP — On-Policy, OFP — Off-Policy, VB — Value-Based, PB — Policy-Based.

2. Overlapping Testing Objectives: Overlapping objectives in testing RL systems complicate evaluations but may not directly hinder the detection of bugs or the verification of functional requirements. Developing a standardized taxonomy and formal definitions could establish clear standards and support the growth of testing RL systems as a field of study. This may

enhance consistency, provide a foundation for future advancements, and clarify the relationships between objectives, such as resilience and robustness.

3. Specific Limitations of Testing Approaches: Each testing approach has limitations. Metamorphic testing relies on manually designed relations, which limits its generalization across

Table 13

Comparative summary of test coverage and adequacy methods in RL.

Reference	Coverage type	Test target	Required inputs/data	Output metric	Test strategy	Limitation (s)	Key insight/conclusion
Trujillo et al. (2020)	Neuron coverage	Neuron activation in DRL	Internal neuron states	Neuron coverage % vs. reward	White-box (coverage analysis)	Weak correlation to performance	Neuron coverage alone is insufficient
Tappler et al. (2022)	State/action coverage (DFS)	State/action space	Traces (DFS)	Coverage ratio of actions/states	Black-box + DFS	Needs trace generation	Effective state/action coverage tool
Tappler and Aichernig (2023)	Observation coverage	Abstract safety states	State abstraction	Coverage of safety-relevant states	White-box + abstraction	Relies on abstraction accuracy	New metric for evaluating abstract safety
Li et al. (2023)	Cluster/state diversity	State clusters	State embeddings	Cluster visitation count	Black-box	Cluster definition sensitivity	Tracks behavior generalization
Lu et al. (2022) and Varshosaz et al. (2023)	Mutation score (faults)	Agent faults	Mutated environments	Mutants detected/mutation score	Black-box mutation	Mutation design challenges	Measures fault exposure effectiveness
Mazouni et al. (2024)	Fault diversity	Fault behavior diversity	Fault outputs	Diversity of faults	Black-box evolution	Computational cost	Diversity better than fault count alone
Uesato et al. (2019)	Failure-probability coverage	Failure cases from initial states	Initial state space	Failure probability distribution	Probabilistic prediction	Assumes known initial conditions	Enables failure-focused testing
Shi et al. (2024)	Multi-granularity coverage	State/action/policy levels	Traces at multiple granularity	Coverage at state/action/policy level	Multi-level coverage testing	Complex metric coordination	Comprehensive multi-scale coverage
He et al. (2024)	Multi-objective test coverage	Exploration and crash detection	Trajectory data	Crash count and diversity	Exploration-based fuzzing	No model introspection	Encourages novel crash-inducing paths
Tappler et al. (2024a)	Cluster coverage	Abstract state clusters	Test cluster trace data	Coverage of clustered traces	Cluster-guided fuzz testing	Trace quality dependent	Efficiently groups critical failure cases
Tappler and Lorber (2024)	Neuron Coverage	Policy, State, Action	Trained policy, abstract MDP, test states	NC score, failure count	coverage analysis	NC does not correlate with agent performance or failure exposure	NC is not a valid standalone test adequacy metric in DRL
Tappler and Lorber (2024)	State Importance	Policy, State, Action	Trained policy, test trajectories	Importance scores, failures	coverage analysis	Requires interpretable state space; visualization-based; not applicable to latent models	Importance scores can identify critical test states; branching states expose more failures

tasks (Eniser et al., 2022). Adversarial testing is computationally expensive and may not achieve adequate testing. Mutation testing struggles to capture RL-specific faults, as its effectiveness depends on the complexity of the environment and the knowledge of fault types. Search-based testing is resource-intensive and often oversimplified, which limits its application in complex settings. Fuzzy logic-based testing performs best in simple environments, restricting its applicability to more complex RL systems (Zhang et al., 2024).

- 4. Test Oracles and Their Challenges:** Test oracles verify whether agents perform as expected despite the challenges posed by RL systems' dynamic and stochastic nature. Three papers proposed oracles. Eniser et al. used metamorphic relations to compare performance across state pairs. Varshosaz et al. introduced interpreter-based oracles that align behavior with formal models, and Zhang et al. applied fuzzy logic to assess policy compliance (Eniser et al., 2022; Varshosaz et al., 2023; Zhang et al., 2024).
- 5. Validation Processes:** Existing frameworks for testing RL employ three validation methods. Of the 42 papers proposed testing approach, 23 rely solely on internal validation methods, such as ablation studies or varying configurations of RL systems. Three papers exclusively utilize external validation by referencing previously proposed methods, some of which are not specifically tailored for testing RL. Sixteen papers combine internal and external validation approaches, while one paper incorporates

human validation by comparing an automated oracle with a human-based one (Zhang et al., 2024).

5.6. Rq6. what research gaps exist in testing RL systems?

This section discusses key research gaps in testing RL systems and outlines opportunities for future work.

Testing Objectives. Robustness is the most frequently tested objective, often overlapping with resilience and adaptability. Stress-testing agents in adversarial scenarios are intended to enhance resilience, but they also contribute to robustness. Assessing robustness across environments can overlap with adaptability. This highlights the need for clear, formal definitions to avoid misinterpretation and redundancy, ensuring accurate assessments and consistent benchmarks for testing RL.

Testing RL systems by ensuring they behave as expected while achieving broader performance goals is crucial. Some RL systems require trade-offs between objectives, which is necessary for real-world applications such as autonomous driving, healthcare, and financial systems, where safety may limit performance-enhancing strategies.

Fairness remains an untested objective, yet it is critical for addressing bias in decision-making. Similarly, scalability testing remains underexplored despite its importance for large-scale RL applications, such as robotics and multi-agent systems. Transparency and interpretability testing are crucial for high-stakes domains such as healthcare and

finance. Long-term generalization testing is also needed to assess RL systems in dynamic environments.

Despite progress, comprehensive testing for RL systems remains underdeveloped. Adaptability testing faces challenges when historical data is unavailable or the system complexity increases. Simplified environments often fail to capture the real-world dynamics of safety, and scalability testing requires more attention, given the growing complexity of RL in practical deployments.

Accordingly, we identified the following gaps in testing objectives:

Gap 1

The evaluation of required trade-offs between testing objectives in RL systems remains underexplored.

Gap 2

Lack of fairness, scalability, transparency, interpretability, and generalization testing of RL systems poses significant challenges, especially in real-world applications where these attributes are crucial.

Testing Approaches. Different testing approaches are employed to evaluate RL systems, each with its advantages and limitations. Mutation testing identifies system vulnerabilities by evaluating mutants, while adversarial testing assesses the agent's robustness against adversary manipulations. Efficient bug tracing remains a critical challenge for these methods. Fuzzing, though random, can be enhanced with search-based methods to guide testing in complex systems. Model-based and data-driven approaches are practical when system dynamics are less stochastic and sufficient data is available. However, these approaches rely on generalization from data, which may not be suitable for highly dynamic, real-world environments.

Simulation-based testing is a helpful initial step, but it does not guarantee real-world effectiveness. For example, CARLA, a popular simulation tool for autonomous driving, is widely used to test RL agents. However, it falls short in translating to real-world scenarios, where dynamic weather, human behavior, and varying traffic conditions introduce unpredictability often absent in simulations. Such variables can significantly impact the performance of RL systems, making real-world testing a challenging task.

The analysis also highlights a limited focus on test case reduction and prioritization, with only five papers addressing these strategies, [Ma et al. \(2024\)](#), [He et al. \(2024\)](#), [Mazouni et al. \(2025\)](#), [Pranger et al. \(2024\)](#) and [Tappier and Lorber \(2024\)](#). These techniques are effective for high-dimensional RL systems, enabling efficient testing by maximizing coverage while minimizing the number of test cases.

Current trends in testing RL depend on adversarial testing, fuzzing, and mutation testing methodologies. These traditional software testing techniques have been adapted for testing RL to enhance the effectiveness of testing. However, balancing efficiency and effectiveness in testing complex RL systems remains a challenge. This analysis supports identifying Gap 3.

Gap 3

A limited focus on test case reduction and prioritization strategies in testing RL systems hinders the efficiency of testing across high-dimensional systems.

Targeted RL System Attributes and Testing Strategies. Single-agent systems dominate testing research in RL due to their simplicity, direct results, and the availability of established benchmarks. In contrast, multi-agent systems (MARL) are more complex and resource-intensive, requiring sophisticated testing strategies to address inter-agent coordination, communication, and emergent behaviors in cooperative or competitive settings. These complexities introduce challenges

such as dependencies between agents' behaviors, task allocation, and conflict resolution, making evaluation challenging.

Model-free systems are preferred due to their straightforward implementation and outcomes. However, model-based RL, known for its sample efficiency and faster adaptation in complex scenarios like autonomous drone navigation, remains underexplored in testing. These systems present unique challenges, including ensuring stability, robustness, and generalization, as they adapt more quickly and are prone to issues such as overfitting.

Simulated environments, particularly those from OpenAI Gym, are commonly used for experimentation with the testing framework. However, simulated settings often oversimplify real-world dynamics, leading to discrepancies in performance when RL systems are deployed in high-stakes domains such as autonomous driving or healthcare. For example, factors such as unpredictable obstacles or real-world noise are often overlooked in simulations, which can result in performance degradation. Expanding experiments to include real-world applications is crucial for enhancing reliability and generalizability.

From this discussion, we identified three gaps in the research on testing RL systems.

Gap 4

The predominant focus is on single-agent systems, neglecting the complexities of evaluating multi-agent systems.

Gap 5

The focus on model-free RL systems overlooks the exploration of testing methods for model-based RL, which are essential for effective adaptation in complex scenarios.

Gap 6

The lack of an experiment testing framework on real-world applications, with most studies focusing on simulated settings that may not accurately reflect practical performance in critical domains.

Test Adequacy Criteria in Testing RL Systems. Test adequacy metrics are crucial for evaluating testing sufficiency. Neuron Coverage identifies underutilized network components but often lacks task-specific insights, which could lead to incomplete assessments. Comparatively, Action and State Coverage ensures exploration across state-action spaces, providing broader insights into system robustness but struggling in complex environments. Observation Coverage, which targets safety-critical states, provides reliability in specific scenarios, while State Diversity Coverage enhances failure detection by promoting diverse exploration. Metrics such as Mutation Score and Fault Diversity Adequacy quantify test effectiveness and comprehensiveness, although they may overlook context-specific nuances.

Among the comparative insights, studies demonstrate that Multi-Granularity Coverage, which measures system aspects at varying levels, offers a more detailed understanding of interactions but demands more computational resources. Conversely, Cluster Coverage focuses on high-priority states, providing efficiency but potentially overlooking broader system behaviors. Comparisons reveal trade-offs in using metrics like Failure Probability-Based Coverage, which excels in evaluating initial conditions but may miss dynamic failures.

These comparisons highlight the importance of striking a balance between depth and efficiency in RL adequacy metrics. For instance, studies suggest that Multi-Objective Test Coverage often struggles to balance between exploration and fault detection. Metrics like state-driven coverage provide more context-specific evaluation, but they require task-specific data to be effective. Ensuring adequate testing is

crucial due to the nature of the RL system. RL systems operate in dynamic environments with unpredictable outcomes, making it essential to use comprehensive adequacy metrics that account for variability and complexity. High-dimensional state and action spaces further challenge traditional testing methods, necessitating the development of tailored approaches to ensure confidence in test results.

Future research could focus on hybrid approaches that combine metrics, allowing for adaptive and context-aware evaluations. Comparative studies are necessary to assess the effectiveness of metrics across various environments and tasks. Two key gaps in testing adequacy have been identified.

Gap 7

The current neuron coverage metrics are general and do not specifically address critical or high-reward states in RL systems.

Gap 8

Enhancements and integration of coverage metrics are necessary to overcome the limitations and achieve better test adequacy in testing RL systems, ensuring comprehensive evaluation of agents in diverse scenarios.

Test Oracles While current test oracles show effectiveness, they have limitations: metamorphic relations risk incomplete testing due to task-specific designs (Eniser et al., 2022), interpreter-based oracles rely on predefined outcomes (Varshosaz et al., 2023), and fuzzy logic struggles in complex environments (Zhang et al., 2024). An oracle adapted to evolving RL settings remains a challenge. Designing oracles that address continuous adaptation and non-deterministic behavior in RL systems is an ongoing challenge.

Gap 9

The need for oracles that provide comprehensive and adaptable testing for RL systems, capable of adjusting to dynamic agent behaviors, environmental changes, and evolving conditions, remains underexplored.

Gap 10

Limited efforts have been made to explore and refine metamorphic relations for testing RL systems.

Validation Processes Internal validation offers detailed insights but may lack broader applicability. While external validation improves generalization, it often overlooks RL-specific aspects, as the models used, such as MDPFuzz (Pang et al., 2022), are not specifically tailored for testing RL. Combining both provides a balanced perspective. Human validation adds reliability but introduces potential subjectivity. Incorporating real-world applications into these validation methods could enhance their robustness and practical relevance.

Gap 11

The lack of RL-specific testing benchmarks leads to reliance on general external validation frameworks, which often overlook RL-specific behaviors and limit evaluation adequacy.

5.7. Rq7. what are the potential future research directions for enhancing testing RL systems?

This subsection discusses potential solutions and research opportunities for the identified eleven gaps. While some solutions directly address these gaps, others suggest areas for further exploration.

1. **Need to Test Trade-offs Between System Objectives:** Evaluating trade-offs between testing objectives, such as safety and performance, remains underexplored. Developing testing methodologies, such as adaptive adjustments or hybrid approaches with weighted priorities, could enhance testing. Research may explore multi-objective testing techniques to manage these trade-offs effectively, particularly through experimentation in domain-specific applications such as healthcare and autonomous driving.
2. **Lack of Adequate Testing for Fairness, Scalability, Transparency, Interpretability, and Generalization:** Testing these objectives is critical yet underexplored. Incorporating them into verification frameworks through techniques such as explainability methods and fairness audits could enhance the comprehensiveness of testing. Research may explore the development of strategies for testing fairness, transparency, and generalization of an RL system. For instance, fairness testing is valuable in hiring algorithms to ensure unbiased decisions; transparency is required in healthcare for justifying medical recommendations, and generalization testing is crucial for autonomous driving to handle diverse and dynamic conditions. Proposing testing approaches for these objectives could focus on applications where they have the most significant impact.
3. **Gap in Test Case Reduction and Prioritization for Efficient Testing:** Research on testing RL systems is still in its early stages, primarily focused on exploring techniques for bug detection and functionality verification. As a result, optimization has not yet become a central focus. However, efficiency in testing, particularly for high-dimensional and complex RL systems, is as important as effectiveness. Incorporating test case reduction and prioritization methods into testing frameworks could address this gap and shift testing from exhaustive, impractical strategies to more feasible and efficient approaches.
4. **Scarcity of Research in Testing Multi-Agent RL Systems:** Multi-agent systems require testing additional aspects not found in single-agent systems. These include coordination, competition, and communication between agents. Research opportunities may focus on methods to assess interactions and emergent behaviors in multi-agent systems, particularly in competitive and cooperative tasks such as robotics or autonomous vehicle applications.
5. **Testing Gap for Model-Based RL Systems:** Model-based RL systems are more robust than model-free ones due to their reliance on an internal model of the environment. This robustness may make some testing approaches that are effective for model-free RL less suitable for model-based systems, as these methods often overlook the environment model's role in guiding decisions and mitigating failures. Testing model-based RL systems using current testing RL approaches presents an opportunity to validate or challenge this hypothesis. Black-box testing could be effective since it focuses solely on inputs and outputs, bypassing the need to analyze internal components. Research opportunities include addressing the unique challenges of testing model-based systems. Testing methodologies consider the environment model's components (such as white-box testing or hybrid approaches) to identify potential vulnerabilities better and improve testing effectiveness for model-based systems.
6. **Deficiency in Validating Testing Approach on Real-World Applications:** Most reviewed studies rely on simplified simulated environments that may not fully reflect real-world performance. However, validating testing approaches in real-world applications, beyond benchmarking, guarantees their practical relevance and applicability. Research may explore bridging the gap between simulated and real-world application testing by addressing scalability and resource constraints and transforming real-world applications from prior studies (e.g., Li et al., 2022a for patient monitoring) into domain-specific scenarios that could enhance the applicability of testing frameworks.

7. **Insufficient Exploration of Neuron Coverage in Testing DRL:** Neuron coverage, investigated in only two studies in testing RL (Trujillo et al., 2020; Tappler and Lorber, 2024), remains a general metric that does not differentiate between neurons activated during high-reward or safety-critical states and those in less significant scenarios. Enhancing these metrics by incorporating task-specific contexts and prioritizing neurons active during critical states could improve test adequacy in DRL systems. Research opportunities include exploring the relationship between neuron activations and scenarios.
8. **Need for Investigating Coverage Metrics Integration in Testing RL Systems:** Combining coverage metrics, such as state-action coverage and reward coverage, into testing practices could improve the thoroughness of evaluations by capturing critical interactions and outcomes more effectively. Research might focus on developing novel metrics tailored to RL environments, enabling testing frameworks that address the variability and complexity of RL systems.
9. **Need for Adaptable Testing Oracles:** Developing adaptable oracles to address dynamic agent behaviors and evolving environments in RL systems remains an open challenge. Research could explore strategies such as real-time learning, predictive modeling, and adaptive feedback to enhance the effectiveness of oracles.
10. **Limited Exploration of Metamorphic Relations in Testing RL:** Metamorphic relations remain underutilized in testing RL despite their potential to detect bugs and evaluate correctness in systems without explicit oracles. Expanding efforts toward designing domain-specific metamorphic relations could improve their applicability to RL environments, thereby enhancing overall testing outcomes. Additionally, research might explore automated methods to generate and validate these relations.
11. **Lack of Standardized Benchmark Frameworks for Testing RL:** The lack of standardized benchmark frameworks for testing RL limits the reliability of validation. Current validation methods often rely on random baselines or testing machine learning frameworks (not tailored for testing reinforcement learning systems). Developing comprehensive frameworks that address RL-specific challenges could enable consistent comparisons and improve testing practice. Research may explore designing benchmarks to capture the behaviors of RL systems across diverse environments, agents, and tasks. (See Table 14 for summary)

6. Threats to validity

This section outlines threats to validity and their corresponding mitigation strategies.

Construct validity may be affected by inconsistent terminologies, such as the interchangeable use of “robustness testing” and “adversarial testing”. Differences in how testing objectives are operationalized make direct comparisons difficult. We address this by providing a dedicated section for testing objectives and another for testing approaches, clearly defining the scope and usage of each term. The complexity of RL systems increases the risk of overlooking important aspects. To mitigate this, we structured the review with dedicating subsections for core testing components (e.g., generation, oracle, adequacy, objectives, repair/debugging), which helped ensure comprehensive coverage. Given the variability in RL systems and experimental settings, we analyzed testing approaches rather than directly comparing results. This reflects the absence of a unified or standard testing setup in the literature. Studies differ in tasks, algorithms, and metrics, making statistical comparisons infeasible. Instead, we compare techniques, methodology, applicability, and limitations.

Internal validity may be impacted by bias in study selection. Filtering papers based on titles, abstracts, and keywords may overlook

relevant studies that use different terminology. To address this, we supplemented our search with snowballing, using references in selected papers to identify additional relevant studies that our original search terms may have missed.

The validity of our conclusions may be compromised by the fact that our findings are based on 49 studies from five digital libraries. While we applied inclusion and exclusion criteria, this may have resulted in the omission of some studies. Given the recency and narrow scope of the field, we adopted flexible criteria to avoid excluding potentially valuable work. Our conclusions are not statistical but based on identifying gaps, trends, and underexplored aspects related to RL system types, learning paradigms, and testing objectives. Studies often lack some information. To avoid misinterpretation, we rely strictly on explicitly stated information. During data extraction, if a required attribute was not explicitly mentioned, we left it blank rather than inferring, thus reducing bias and wrong information.

External validity concerns the generalizability of our findings across RL applications. Testing methodologies vary across domains, techniques, and objectives. Our review explicitly focuses on testing RL systems themselves rather than integrated systems (e.g., autonomous vehicles), which include RL as a subsystem alongside others. We limited our scope to RL systems that contain basic learning components at the unit or system levels. Acceptance testing and deployment-level validation of applications are beyond the scope of this review. Additionally, most reviewed studies conducted experiments on simulated or benchmark tasks, which further limits their generalizability. While we aimed for broad coverage, the findings may not apply to all RL-integrated systems.

Temporal bias may also be present, as all reviewed studies were published from 2013 onwards. RL testing is a relatively new field. We actively monitored new publications throughout the review process to ensure the study remained up to date. Nonetheless, future developments may extend beyond the scope of this review.

These threats are consistent with known challenges in systematic literature review research, including misinterpretation of concepts, bias in study selection, generalization issues, and reliability of synthesis, as discussed in Zhou et al. (2016). Our methodology adheres to established practices to mitigate these threats, ensuring that our findings are as reliable and representative as possible.

7. Conclusions

This paper presents the first comprehensive literature review on testing RL systems. It provides an in-depth overview of achievements, challenges, and gaps to guide future research. Through a structured search strategy, we identified 49 relevant studies. Data was extracted using a standardized form to analyze contributions, testing techniques, RL models, objectives, and limitations, ensuring thorough synthesis.

This study answers seven research questions, covering testing objectives, methodologies, adequacy metrics, challenges, and future opportunities. We identified eleven gaps in the research on testing RL systems, including the lack of validating testing RL frameworks on real-world applications and underexplored areas such as fairness and generalization testing.

Significant progress in testing RL has been observed, particularly since 2019, with a greater focus on robustness and safety. Traditional software testing methods, including mutation and search-based, have been adapted for testing RL. Hybrid testing methods, which integrate adversarial testing and simulation techniques, show promise in enhancing testing effectiveness. Additionally, test adequacy metrics such as state diversity and action coverage are primary in the success of testing RL. Emerging trends highlight the importance of test oracles, machine learning- and search-based testing tools, and the expansion of testing frameworks that support multi-agent systems and real-world applications.

The evolving field of testing RL presents unique challenges. While progress has been made, testing RL requires further efforts to ensure systems are trustworthy and deployable in high-stake domains. Future research should prioritize addressing identified gaps.

Table 14

Key gaps in testing RL and suggested mitigations.

No	Gap	Mitigation
1	The evaluation of required trade-offs between testing objectives in RL systems remains underexplored.	• Conduct multi-objective testing. • Develop adaptive adjustments or hybrid testing.
2	Lack of fairness, scalability, transparency, interpretability, and generalization testing of RL systems poses significant challenges, especially in real-world applications where these attributes are crucial.	• Propose frameworks to test RL systems for these objectives. • Test RL applications where an objective is most impactful.
3	Limited focus on test case reduction and prioritization strategies in testing RL systems, hindering the efficiency of the testing across high-dimensional systems.	• Incorporate test case reduction and prioritization techniques into testing frameworks.
4	The predominant focus is on single-agent systems, neglecting the complexities of multi-agent system evaluations.	• Propose methodologies for testing multi-agent-based RL systems' specific components and required functionalities.
5	The focus on model-free systems, neglecting the exploration of model-based methods that are essential for effective adaptation in complex scenarios.	• Develop tests that address the characteristics of model-based RL systems. • Investigate the applicability of existing RL testing methods to model-based RL systems.
6	The lack of evaluations in real-world applications, with most studies focusing on simulated settings that may not reflect practical performance in critical domains.	• Study how to bridge the gap between simulated and real-world applications. • Utilize available real data to develop a target RL system.
7	The current neuron coverage metrics are general and do not specifically address critical or high-reward states in RL systems.	• Investigate the relationship between neuron coverage and the activation of critical or high-reward states.
8	Enhancements and integration of coverage metrics are necessary to overcome the limitations and achieve better test adequacy in RL testing, ensuring comprehensive evaluation of agents in diverse scenarios.	• Enhance existing coverage metrics or develop new ones, ensuring their applicability to different RL system types and testing objectives for improved test adequacy.
9	The need for oracles that provide comprehensive and adaptable testing for RL systems, capable of adjusting to dynamic agent behaviors, environmental changes, and evolving conditions, remains underexplored.	• Develop and explore adaptable testing oracles for RL systems. • Investigate methods for designing oracles that adjust to dynamic agent behaviors, environmental changes, and evolving conditions.
10	Limited efforts have been made to explore and refine metamorphic relations to enhance testing RL systems.	• Design a broad set of metamorphic relations informed by human expertise.
11	The absence of standardized benchmark frameworks for testing RL leads to a reliance on external validation approaches that may overlook the unique characteristics of the RL systems being assessed, limiting effective evaluation.	• Propose a benchmark framework with a suite of RL systems for validating RL testing frameworks. • Develop and release RL testing frameworks with available code to enable research and benchmarking.

CRediT authorship contribution statement

Amal Sunba: Writing – review & editing, Writing – original draft, Visualization, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Jameleddine Hassine:** Writing – review & editing, Supervision, Resources, Methodology. **Moataz Ahmed:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Investigation.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used ChatGPT in order to proofread and correct grammar and spelling. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to acknowledge the support of King Fahd University of Petroleum and Minerals (KFUPM), Saudi Arabia, the Interdisciplinary Research Center for Intelligent Secure Systems (IRC-ISS) at KFUPM, and the SDAIA-KFUPM Joint Research Center for Artificial Intelligence (JRC-AI) in the development of this work.

Data availability

No data was used for the research described in the article.

References

- Allen, M., Pearn, K., Monks, T., 2021. Developing an OpenAI gym-compatible framework and simulation environment for testing deep reinforcement learning agents solving the ambulance location problem. CoRR [abs/2101.04434](https://arxiv.org/abs/2101.04434), arXiv:2101.04434, URL: <https://arxiv.org/abs/2101.04434>.
- AlMahamid, F., Grolinger, K., 2022. Reinforcement learning algorithms: An overview and classification. <https://dx.doi.org/10.48550/ARXIV.2209.14940>, CoRR [abs/2209.14940](https://arxiv.org/abs/2209.14940), arXiv:2209.14940.
- Bacci, E., Parker, D., 2020. Probabilistic guarantees for safe deep reinforcement learning. In: Bertrand, N., Jansen, N. (Eds.), Formal Modeling and Analysis of Timed Systems - 18th International Conference, FORMATS 2020, Vienna, Austria, September 1-3, 2020, Proceedings. In: Lecture Notes in Computer Science, vol. 12288, Springer, pp. 231–248. https://dx.doi.org/10.1007/978-3-030-57628-8_14.
- Behzadan, V., Hsu, W.H., 2019. RL-based method for benchmarking the adversarial resilience and robustness of deep reinforcement learning policies. In: Romanovsky, A.B., Troubitsyna, E., Gashi, I., Schoitsch, E., Bitsch, F. (Eds.), Computer Safety, Reliability, and Security - SAFECOMP 2019 Workshops, ASSURE, DECSos, SASSUR, STRIVE, and WAISE, Turku, Finland, September 10, 2019, Proceedings. In: Lecture Notes in Computer Science, vol. 11699, Springer, pp. 314–325. https://dx.doi.org/10.1007/978-3-030-26250-1_25.
- Beizer, B., 1990. Software Testing Techniques, second ed. Van Nostrand Reinhold.
- Bellemare, M.G., Naddaf, Y., Veness, J., Bowling, M., 2013. The arcade learning environment: An evaluation platform for general agents. J. Artificial Intelligence Res. 47, 253–279. <https://dx.doi.org/10.1613/JAIR.3912>.
- Biagiola, M., Tonella, P., 2022. Testing the plasticity of reinforcement learning-based systems. ACM Trans. Softw. Eng. Methodol. 31 (4), 80:1–80:46. <https://dx.doi.org/10.1145/3511701>.

- Biagiola, M., Tonella, P., 2024. Testing of deep reinforcement learning agents with surrogate models. *ACM Trans. Softw. Eng. Methodol.* 33 (3), 73:1–73:33. <http://dx.doi.org/10.1145/3631970>.
- Casper, S., Hadfield-Menell, D., Kreiman, G., 2023. White-box adversarial policies in deep reinforcement learning. In: Pedroza, G., Huang, X., Chen, X.C., Theodorou, A., Hernández-Orallo, J., Castillo-Effen, M., Mallah, R., McDermid, J.A. (Eds.), *Proceedings of the Workshop on Artificial Intelligence Safety 2023 (SafeAI 2023)* Co-located with the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI 2023), Washington DC, USA, February 13–14, 2023. In: *CEUR Workshop Proceedings*, vol. 3381, CEUR-WS.org, URL: <https://ceur-ws.org/Vol-3381/43.pdf>.
- Che, H., Bai, Z., Zuo, R., Li, H., 2020. A deep reinforcement learning approach to the optimization of data center task scheduling. *Complex.* 2020, 3046769:1–3046769:12. <http://dx.doi.org/10.1155/2020/3046769>.
- Chen, J., Wang, Y., Wang, J., Xie, X., Wang, D., Wang, Q., Xu, F., 2025. Demo2Test: Transfer testing of agent in competitive environment with failure demonstrations. *ACM Trans. Softw. Eng. Methodol.* 34 (2), <http://dx.doi.org/10.1145/3696001>.
- Chen, Z., Zhang, J.M., Hort, M., Harman, M., Sarro, F., 2024. Fairness testing: A comprehensive survey and analysis of trends. *ACM Trans. Softw. Eng. Methodol.* 33 (5), 137:1–137:59. <http://dx.doi.org/10.1145/3652155>.
- Corsi, D., Marchesini, E., Farinelli, A., 2020. Evaluating the safety of deep reinforcement learning models using semi-formal verification. *CoRR abs/2010.09387*, arXiv:2010.09387, URL: <https://arxiv.org/abs/2010.09387>.
- Daoun, D., Ibat, F., Alom, M.Z., Aung, Z., Azim, M.A., 2021. Reinforcement learning: A friendly introduction. In: Awan, I., Benbernou, S., Younas, M., Aleksey, M. (Eds.), *The International Conference on Deep Learning, Big Data and Blockchain (Deep-BDB 2021)*, Virtual Event, 23–25 August, 2021. In: *Lecture Notes in Networks and Systems*, vol. 309, Springer, pp. 134–146. http://dx.doi.org/10.1007/978-3-030-84337-3_11.
- Dimitriu, A., Michaletzky, T.V., Remeli, V., Tihanyi, V., 2024. A reinforcement learning approach to military simulations in command: Modern operations. *IEEE Access* 12, 77501–77513. <http://dx.doi.org/10.1109/ACCESS.2024.3406148>.
- Dosovitskiy, A., Ros, G., Codevilla, F., López, A.M., Koltun, V., 2017. CARLA: An open urban driving simulator. In: 1st Annual Conference on Robot Learning, CoRL 2017, Mountain View, California, USA, November 13–15, 2017, *Proceedings. In: Proceedings of Machine Learning Research*, vol. 78, PMLR, pp. 1–16, URL: <http://proceedings.mlr.press/v78/dosovitskiy17a.html>.
- Dulac-Arnold, G., Levine, N., Mankowitz, D.J., Li, J., Paduraru, C., Goyal, S., Hester, T., 2021. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Mach. Learn.* 110 (9), 2419–2468. <http://dx.doi.org/10.1007/S10994-021-05961-4>.
- Eniser, H.F., Gros, T.P., Wüstholtz, V., Hoffmann, J., Christakis, M., 2022. Metamorphic relations via relaxations: an approach to obtain oracles for action-policy testing. In: Ryu, S., Smaragdakis, Y. (Eds.), *ISSTA '22: 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, Virtual Event, South Korea, July 18–22, 2022. *ACM*, pp. 52–63. <http://dx.doi.org/10.1145/3533767.3534392>.
- FaramaFoundation, 2024. Gymnasium toy text environments. URL: https://gymnasium.farama.org/environments/toy_text/.
- Foundation, O.G., 2022. Humanoid. URL: <https://www.gymnasium.dev/environments/mujoco/humanoid/> (Accessed 22 October 2024).
- GitHub, 2016. TORCS. URL: https://github.com/ugo-nama-kun/gym_torcs/ (Accessed 22 October 2024).
- Gross, D., Jansen, N., Junges, S., Pérez, G.A., 2022. COOL-MC: A comprehensive tool for reinforcement learning and model checking. In: Dong, W., Talpin, J. (Eds.), *Dependable Software Engineering. Theories, Tools, and Applications - 8th International Symposium, SETTA 2022, Beijing, China, October 27–29, 2022*, *Proceedings. In: Lecture Notes in Computer Science*, vol. 13649, Springer, pp. 41–49. http://dx.doi.org/10.1007/978-3-031-21213-0_3.
- Gross, D., Spieker, H., 2024. Probabilistic model checking of stochastic reinforcement learning policies. In: Rocha, A.P., Steels, L., van den Herik, H.J. (Eds.), *Proceedings of the 16th International Conference on Agents and Artificial Intelligence, ICAART 2024, Volume 3, Rome, Italy, February 24–26, 2024*. SCITEPRESS, pp. 438–445. <http://dx.doi.org/10.5220/0012357700003636>.
- Guo, J., Chen, Y., Hao, Y., Yin, Z., Yu, Y., Li, S., 2022. Towards comprehensive testing on the robustness of cooperative multi-agent reinforcement learning. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops 2022, New Orleans, la, USA, June 19–20, 2022*. IEEE, pp. 114–121. <http://dx.doi.org/10.1109/CVPRW56347.2022.00022>.
- Gym, O., 2022a. Acrobot. URL: https://www.gymnasium.dev/environments/classic_control/acrobot/#acrobot (Accessed 22 October 2024).
- Gym, O., 2022b. Bipedal walker. URL: https://www.gymnasium.dev/environments/box2d/bipedal_walker/ (Accessed 22 October 2024).
- Gym, O., 2022c. Humanoid standup. URL: https://www.gymnasium.dev/environments/mujoco/humanoid_standup/#humanoid-standup (Accessed 22 October 2024).
- Gym, O., 2022d. Pendulum. URL: https://www.gymnasium.dev/environments/classic_control/pendulum/#pendulum (Accessed 22 October 2024).
- Gym, O., 2022e. Taxi. URL: https://www.gymnasium.dev/environments/toy_text/taxi/ (Accessed 22 October 2024).
- He, J., Yang, Z., Shi, J., Yang, C., Kim, K., Xu, B., Zhou, X., Lo, D., 2024. Curiosity-driven testing for sequential decision-making process. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14–20, 2024*. *ACM*, pp. 165:1–165:14. <http://dx.doi.org/10.1145/3597503.3639149>.
- Hetzl, B., 1988. *The Complete Guide to Software Testing*, second ed. QED Information Sciences, Inc., USA.
- Hoen, P.J., Tuyls, K., Panait, L., Luke, S., Poutré, J.A.L., 2005. An overview of cooperative and competitive multiagent learning. In: Tuyls, K., Hoen, P.J., Verbeeck, K., Sen, S. (Eds.), *Learning and Adaption in Multi-Agent Systems*, First International Workshop, LAMAS 2005, Utrecht, the Netherlands, July 25, 2005, *Revised Selected Papers. In: Lecture Notes in Computer Science*, vol. 3898, Springer, pp. 1–46. http://dx.doi.org/10.1007/11691839_1.
- Hu, M., Zhang, J., Matkovic, L., Liu, T., Yang, X., 2022. Reinforcement learning in medical image analysis: Concepts, applications, challenges, and future directions. <http://dx.doi.org/10.48550/ARXIV.2206.14302>, *CoRR abs/2206.14302*, arXiv:2206.14302.
- Huang, X., Kroening, D., Ruan, W., Sharp, J., Sun, Y., Thamo, E., Wu, M., Yi, X., 2020. A survey of safety and trustworthiness of deep neural networks: Verification, testing, adversarial attack and defence, and interpretability. *Comput. Sci. Rev.* 37, 100270. <http://dx.doi.org/10.1016/J.COSREV.2020.100270>.
- IEEE, 1983. IEEE standard glossary of software engineering terminology. In: *ANSI / IEEE Std 729- 1983*. pp. 1–40. <http://dx.doi.org/10.1109/IEEESTD.1983.7435207>.
- Jang, B., Kim, M., Harerimana, G., Kim, J.W., 2019. Q-learning algorithms: A comprehensive classification and applications. *IEEE Access* 7, 133653–133667. <http://dx.doi.org/10.1109/ACCESS.2019.2941229>.
- Kiran, B.R., Sobh, I., Talpaert, V., Mannion, P., Sallab, A.A.A., Yogamani, S.K., Pérez, P., 2022. Deep reinforcement learning for autonomous driving: A survey. *IEEE Trans. Intell. Transp. Syst.* 23 (6), 4909–4926. <http://dx.doi.org/10.1109/TITS.2021.3054625>.
- Kitchenham, B.A., Charters, S., 2007. Guidelines for performing Systematic Literature Reviews in Software Engineering. Technical Report EBSE 2007-001, Keele University and Durham University, URL: https://www.elsevier.com/_data/promis_misc/525444systematicreviewsguide.pdf.
- Kurach, K., Raichuk, A., Stanczyk, P., Zajac, M., Bachem, O., Espeholt, L., Riquelme, C., Vincent, D., Michalski, M., Bousquet, O., Gelly, S., 2020. Google research football: A novel reinforcement learning environment. In: *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, the Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, the Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7–12, 2020*. AAAI Press, pp. 4501–4510. <http://dx.doi.org/10.1609/AAAI.V34I04.5878>.
- Landers, M., Doryab, A., 2023. Deep reinforcement learning verification: A survey. *ACM Comput. Surv.* 55 (14s), 330:1–330:31. <http://dx.doi.org/10.1145/3596444>.
- Lehmann, M., 2024. The definitive guide to policy gradients in deep reinforcement learning: Theory, algorithms and implementations. <http://dx.doi.org/10.48550/ARXIV.2401.13662>, *CoRR abs/2401.13662*, arXiv:2401.13662.
- Leurent, E., 2018. Highway: An environment for autonomous driving decision-making. URL: <https://github.com/eleurent/highway-env> (Accessed 22 October 2024).
- Li, M., Shi, C., Wu, Z., Fryzlewicz, P., 2022a. Testing stationarity and change point detection in reinforcement learning. arXiv preprint [arXiv:2203.01707](https://arxiv.org/abs/2203.01707).
- Li, T., Wan, X., Özbek, M.M., 2022b. AgentFuzz: Fuzzing for deep reinforcement learning systems. In: *IEEE International Symposium on Software Reliability Engineering Workshops, ISSRE 2022 - Workshops, Charlotte, NC, USA, October 31 - Nov. 3, 2022*. IEEE, pp. 110–113. <http://dx.doi.org/10.1109/ISSREW55968.2022.00049>.
- Li, Z., Wu, X., Zhu, D., Cheng, M., Chen, S., Zhang, F., Xie, X., Ma, L., Zhao, J., 2023. Generative model-based testing on decision-making policies. In: *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11–15, 2023*. IEEE, pp. 243–254. <http://dx.doi.org/10.1109/ASE56229.2023.00153>.
- Li, J., Zheng, Z., Du, X., Wang, H., Liu, Y., 2025. DRLMutation: A comprehensive framework for mutation testing in deep reinforcement learning systems. *ACM Trans. Softw. Eng. Methodol.* <http://dx.doi.org/10.1145/3721978>, Just Accepted.
- Liu, Q., Lee, X.Y., Sarkar, S., 2024. A study of natural robustness of deep reinforcement learning algorithms towards adversarial perturbations. *AI Open* 5, 126–141. <http://dx.doi.org/10.1016/J.AIOPEN.2024.08.005>.
- Liu, M., Shen, X., Pan, W., 2022. Deep reinforcement learning for personalized treatment recommendation. *Stat. Med.* 41 (20), 4034–4056.
- Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., Mordatch, I., 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. In: Guyon, I., von Luxburg, U., Bengio, S., Wallach, H.M., Fergus, R., Vishwanathan, S.V.N., Garnett, R. (Eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA*. pp. 6379–6390, URL: <https://proceedings.neurips.cc/paper/2017/hash/68a9750337a418a86fe06c1991a1d64c-Abstract.html>.
- Lu, Y., Sun, W., Sun, M., 2022. Towards mutation testing of reinforcement learning systems. *J. Syst. Archit.* 131, 102701. <http://dx.doi.org/10.1016/J.SYSARC.2022.102701>.

- Ma, X., Wang, Y., Wang, J., Xie, X., Wu, B., Li, S., Xu, F., Wang, Q., 2024. Enhancing multi-agent system testing with diversity-guided exploration and adaptive critical state exploitation. In: Christakis, M., Pradel, M. (Eds.), Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024. ACM, pp. 1491–1503. <http://dx.doi.org/10.1145/3650212.3680376>.
- Marijan, D., Godlieb, A., 2020. Software testing for machine learning. In: The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, the Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, the Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020. AAAI Press, pp. 13576–13582. <http://dx.doi.org/10.1609/AAAI.V34I09.7084>.
- Marston, M., Baca, G., 2015. ACAS-Xu Initial Self-Separation Flight Tests. Technical Report DFR-EC-DAA-TN22968, NASA Armstrong Flight Research Center, Edwards AFB, CA, United States, URL: <https://ntrs.nasa.gov/api/citations/20150008347/downloads/20150008347.pdf>, Public Use Permitted.
- Mathieu, T., Della Vecchia, R., Shilova, A., Centa, M.M., Kohler, H., Maillard, O.-A., Preux, P., 2023. AdaStop: adaptive statistical testing for sound comparisons of Deep RL agents. arXiv preprint [arXiv:2306.10882](https://arxiv.org/abs/2306.10882).
- Mazouni, Q., Spieker, H., Godlieb, A., Acher, M., 2024. Testing for fault diversity in reinforcement learning. In: Lonetti, F., Guerriero, A., Saadatmand, M., Budnik, C.J., Li, J. (Eds.), Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024), Lisbon, Portugal, April 15-16, 2024. ACM, pp. 136–146. <http://dx.doi.org/10.1145/3644032.3644458>.
- Mazouni, Q., Spieker, H., Godlieb, A., Acher, M., 2025. Policy testing with MDPFuzz (replicability study). In: Koziolek, A., Lamprecht, A., Thüm, T., Burger, E. (Eds.), Software Engineering 2025, Fachtagung des GI-Fachbereichs Softwaretechnik, Karlsruhe, Germany, February 24-28, 2025. In: LNI, vol. P-360, Gesellschaft für Informatik e.V., p. 10. <http://dx.doi.org/10.18420/SE2025-10>.
- Moerland, T.M., Broekens, J., Ploaat, A., Jonker, C.M., 2023. Model-based reinforcement learning: A survey. Found. Trends Mach. Learn. 16 (1), 1–118. <http://dx.doi.org/10.1561/22000000086>.
- Morán, J., Bertolino, A., de la Riva, C., Tuya, J., 2023. Fault localization for reinforcement learning. In: IEEE International Conference on Artificial Intelligence Testing, AITest 2023, Athens, Greece, July 17-20, 2023. IEEE, pp. 49–50. <http://dx.doi.org/10.1109/AITEST58265.2023.00016>.
- Mqirmi, P.E., Belardinelli, F., León, B.G., 2021. An abstraction-based method to check multi-agent deep reinforcement-learning behaviors. In: Dignum, F., Lomuscio, A., Endriss, U., Nowé, A. (Eds.), AAMAS '21: 20th International Conference on Autonomous Agents and Multiagent Systems, Virtual Event, United Kingdom, May 3-7, 2021. ACM, pp. 474–482. <http://dx.doi.org/10.5555/3463952.3464012>, URL: <https://www.ifaaamas.org/Proceedings/aamas2021/pdfs/p474.pdf>.
- Muskardin, E., Tappler, M., Aichernig, B.K., Pill, I., 2023. Reinforcement learning under partial observability guided by learned environment models. In: Herber, P., Wijs, A. (Eds.), IFM 2023 - 18th International Conference, IFM 2023, Leiden, the Netherlands, November 13-15, 2023. Proceedings. In: Lecture Notes in Computer Science, vol. 14300, Springer, pp. 257–276. http://dx.doi.org/10.1007/978-3-031-47705-8_14.
- Myers, G.J., 2004. The Art of Software Testing, second ed. Wiley, URL: <http://eu.wiley.com/WileyCDA/WileyTitle/productCd-0471469122.html>.
- Nayab, S., Wotawa, F., 2024. Testing and reinforcement learning - A structured literature review. In: 24th IEEE International Conference on Software Quality, Reliability, and Security, QRS - Companion, Cambridge, United Kingdom, July 1-5, 2024. IEEE, pp. 326–335. <http://dx.doi.org/10.1109/QRS-C63300.2024.00049>.
- Nikanjam, A., Morovati, M.M., Khomh, F., Braiek, H.B., 2022. Faults in deep reinforcement learning programs: a taxonomy and a detection approach. Autom. Softw. Eng. 29 (1), 8. <http://dx.doi.org/10.1007/S10515-021-00313-X>.
- OpenAI Gym, 2022a. Ant. Available: <https://www.gymnasium.dev/environments/mujoco/ant/#ant>.
- OpenAI Gym, 2022b. Cartpole. Available: https://www.gymnasium.dev/environments/classic_control/cart_pole/#cart-pole.
- OpenAI Gym, 2022c. Frozen lake. Available: https://www.gymnasium.dev/environments/toy_text/frozen_lake/.
- OpenAI Gym, 2022d. Halfcheetah. Available: https://www.gymnasium.dev/environments/mujoco/half_cheetah/.
- OpenAI Gym, 2022e. Lunarlander. Available: https://www.gymnasium.dev/environments/box2d/lunar_lander/.
- OpenAI Gym, 2022f. Mountain car. Available: https://www.gymnasium.dev/environments/classic_control/mountain_car/#mountain-car.
- OpenAI Gym, 2022g. Pong. Available: <https://www.gymnasium.dev/environments/atari/pong/#pong>.
- OpenAI Gym, 2022h. Swimmer. Available: <https://www.gymnasium.dev/environments/mujoco/swimmer/#swimmer>.
- OpenAI Gym, 2022i. Walker. Available: <https://www.gymnasium.dev/environments/mujoco/walker2d/#walker2d>.
- Pang, Q., Yuan, Y., Wang, S., 2022. MDPFuzz: testing models solving Markov decision processes. In: Ryu, S., Smaragdakis, Y. (Eds.), ISSTA '22: 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, South Korea, July 18 - 22, 2022. ACM, pp. 378–390. <http://dx.doi.org/10.1145/3533767.3534388>.
- Phan, T., Gabor, T., Sedlmeier, A., Ritz, F., Kempter, B., Klein, C., Sauer, H., Schmid, R.N., Wieghardt, J., Zeller, M., Linnhoff-Popien, C., 2020. Learning and testing resilience in cooperative multi-agent systems. In: Seghrouchni, A.E.F., Sukthakar, G., An, B., Yorke-Smith, N. (Eds.), Proceedings of the 19th International Conference on Autonomous Agents and Multiagent Systems, AAMAS '20, Auckland, New Zealand, May 9-13, 2020. International Foundation for Autonomous Agents and Multiagent Systems, pp. 1055–1063. <http://dx.doi.org/10.5555/3398761.3398884>, URL: <https://dl.acm.org/doi/10.5555/3398761.3398884>.
- Pranger, S., Chockler, H., Tappler, M., Könighofer, B., 2024. Test where decisions matter: Importance-driven testing for deep reinforcement learning. In: Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J.M., Zhang, C. (Eds.), Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/317cccd29ed464df181c781cb436180-Abstract-Conference.html.
- Raz, A.K., Nolan, S.M., Levin, W., Mall, K., Mia, A., Mockus, L., Ezra, K., Williams, K., 2022. Test and evaluation of reinforcement learning via robustness testing and explainable AI for high-speed aerospace vehicles. In: 2022 IEEE Aerospace Conference. AERO, pp. 1–14. <http://dx.doi.org/10.1109/AERO53065.2022.9843563>.
- Reuer, K., Landgraf, J., Fösel, T., O'Sullivan, J., Beltrán, L., Akin, A., Norris, G.J., Remm, A., Kerschbaum, M., Besse, J.-C., et al., 2023. Realizing a deep reinforcement learning agent for real-time quantum feedback. Nat. Commun. 14 (1), 7138.
- Rojas, D.J.B., Medina, M.C.C., Fernandes, B.J.T., Barros, P.V.A., 2024. The use of reinforcement learning algorithms in object tracking: A systematic literature review. Neurocomputing 596, 127954. <http://dx.doi.org/10.1016/J.NEUCOM.2024.127954>.
- Samvelyan, M., Rashid, T., de Witt, C.S., Farquhar, G., Nardelli, N., Rudner, T.G.J., Hung, C., Torr, P.H.S., Foerster, J.N., Whiteson, S., 2019. The StarCraft multi-agent challenge. In: Elkind, E., Veloso, M., Agmon, N., Taylor, M.E. (Eds.), Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS '19, Montreal, QC, Canada, May 13-17, 2019. International Foundation for Autonomous Agents and Multiagent Systems, pp. 2186–2188, URL: <http://dl.acm.org/citation.cfm?id=3332052>.
- Shi, Y., Yin, B., Zheng, Z., 2024. Multi-granularity coverage criteria for deep reinforcement learning systems. J. Syst. Softw. 212, 112016. <http://dx.doi.org/10.1016/J.JSS.2024.112016>.
- Sutton, R.S., Barto, A.G., 1998. Reinforcement learning - an introduction. Adaptive computation and machine learning, MIT Press, URL: <https://www.worldcat.org/oclc/37293240>.
- Talendar, 2021. Flappy bird. Available: <https://github.com/Talendar/flappy-bird-gym>.
- Tambon, F., Majdinasab, V., Nikanjam, A., Khomh, F., Antoniol, G., 2023. Mutation testing of deep reinforcement learning based on real faults. In: IEEE Conference on Software Testing, Verification and Validation, ICST 2023, Dublin, Ireland, April 16-20, 2023. IEEE, pp. 188–198. <http://dx.doi.org/10.1109/ICST57152.2023.00026>.
- Tappler, M., Aichernig, B.K., 2023. Differential safety testing of deep RL agents enabled by automata learning. In: Steffen, B. (Ed.), Bridging the Gap Between AI and Reality - First International Conference, AISoLA 2023, Crete, Greece, October 23-28, 2023. Proceedings. In: Lecture Notes in Computer Science, vol. 14380, Springer, pp. 138–159. http://dx.doi.org/10.1007/978-3-031-46002-9_8.
- Tappler, M., Córdoba, F.C., Aichernig, B.K., Könighofer, B., 2022. Search-based testing of reinforcement learning. In: Raedt, L.D. (Ed.), Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23-29 July 2022. ijcai.org, pp. 503–510. <http://dx.doi.org/10.24963/IJCAI.2022/72>.
- Tappler, M., Lorber, F., 2024. Bridging the gap between models in RL: test models vs. Neural networks. In: IEEE International Conference on Software Testing, Verification and Validation, ICST 2024 - Workshops, Toronto, on, Canada, May 27-31, 2024. IEEE, pp. 68–77. <http://dx.doi.org/10.1109/ICSTW60967.2024.00023>.
- Tappler, M., Muskardin, E., Aichernig, B.K., Könighofer, B., 2024a. Learning environment models with continuous stochastic dynamics - with an application to deep RL testing. In: IEEE Conference on Software Testing, Verification and Validation, ICST 2024, Toronto, on, Canada, May 27-31, 2024. IEEE, pp. 197–208. <http://dx.doi.org/10.1109/ICST60714.2024.00026>.
- Tappler, M., Pferscher, A., Aichernig, B.K., Könighofer, B., 2024b. Learning and repair of deep reinforcement learning policies from fuzz-testing data. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024. ACM, pp. 6:1–6:13. <http://dx.doi.org/10.1145/3597503.3623311>.
- Team, P., 2020. Super mario bros. Available at: https://pytorch.org/tutorials/intermediate/mario_rl_tutorial.html.
- Thomas, D., Biagiola, M., Humbatova, N., Wardat, M., Jahangirova, G., Rajan, H., Tonella, P., 2024. muPRL: A mutation testing pipeline for deep reinforcement learning based on real faults. <http://dx.doi.org/10.48550/ARXIV.2408.15150>, CoRR abs/2408.15150, arXiv:2408.15150.
- Tosch, E., Clary, K., Foley, J., Jensen, D.D., 2019. Toybox: A suite of environments for experimental evaluation of deep reinforcement learning. CoRR abs/1905.02825, arXiv:1905.02825, URL: <http://arxiv.org/abs/1905.02825>.

- Trujillo, M., Linares-Vásquez, M., Escobar-Velásquez, C., Dusparic, I., Cardozo, N., 2020. Does neuron coverage matter for deep reinforcement learning?: A preliminary study. In: ICSE '20: 42nd International Conference on Software Engineering, Workshops, Seoul, Republic of Korea, 27 June - 19 July, 2020. ACM, pp. 215–220. <http://dx.doi.org/10.1145/3387940.3391462>.
- Uesato, J., Kumar, A., Szepesvári, C., Erez, T., Ruderman, A., Anderson, K., Dvijotham, K.D., Heess, N., Kohli, P., 2019. Rigorous agent evaluation: An adversarial approach to uncover catastrophic failures. In: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019. OpenReview.net, URL: <https://openreview.net/forum?id=B1xbQhRcK7>.
- Varshosaz, M., Ghaffari, M., Johnsen, E.B., Wasowski, A., 2023. Formal specification and testing for reinforcement learning. *Proc. ACM Program. Lang.* 7 (ICFP), 125–158. <http://dx.doi.org/10.1145/3607835>.
- Wan, X., Li, T., Lin, W., Cai, Y., Zheng, Z., 2024. Coverage-guided fuzzing for deep reinforcement learning systems. *J. Syst. Softw.* 210, 111963. <http://dx.doi.org/10.1016/J.JSS.2024.111963>.
- Wang, H., Wang, J., 2024. Enhancing multi-UAV air combat decision making via hierarchical reinforcement learning. *Sci. Rep.* 14 (1), 4458.
- Zhang, J.M., Harman, M., Ma, L., Liu, Y., 2022. Machine learning testing: Survey, landscapes and horizons. *IEEE Trans. Softw. Eng.* 48 (2), 1–36. <http://dx.doi.org/10.1109/TSE.2019.2962027>.
- Zhang, S., Song, H., Wang, Q., Pei, Y., 2024. Fuzzy logic guided reward function variation: An oracle for testing reinforcement learning programs. <http://dx.doi.org/10.48550/ARXIV.2406.19812>, CoRR [abs/2406.19812](https://arxiv.org/abs/2406.19812), arXiv:2406.19812.
- Zhi, D., Wang, P., Chen, C., Zhang, M., 2024. Robustness verification of deep reinforcement learning based control systems using reward martingales. In: Wooldridge, M.J., Dy, J.G., Natarajan, S. (Eds.), Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20–27, 2024, Vancouver, Canada. AAAI Press, pp. 19992–20000. <http://dx.doi.org/10.1609/AAAI.V38I18.29976>.
- Zhou, X., Jin, Y., Zhang, H., Li, S., Huang, X., 2016. A map of threats to validity of systematic literature reviews in software engineering. In: Potanin, A., Murphy, G.C., Reeves, S., Dietrich, J. (Eds.), 23rd Asia-Pacific Software Engineering Conference, APSEC 2016, Hamilton, New Zealand, December 6–9, 2016. IEEE Computer Society, pp. 153–160. <http://dx.doi.org/10.1109/APSEC.2016.031>.
- Zhou, Z., Liu, G., 2023. Robustness testing for multi-agent reinforcement learning: State perturbations on critical agents. In: Gal, K., Nowé, A., Nalepa, G.J., Fairstein, R., Radulescu, R. (Eds.), ECAI 2023 - 26th European Conference on Artificial Intelligence, September 30 - October 4, 2023, Kraków, Poland - Including 12th Conference on Prestigious Applications of Intelligent Systems. PAIS 2023, In: *Frontiers in Artificial Intelligence and Applications*, vol. 372, IOS Press, pp. 3131–3139. <http://dx.doi.org/10.3233/FAIA230632>.
- Zolfagharian, A., Abdellatif, M., Briand, L.C., Bagherzadeh, M., S., R., 2023. A search-based testing approach for deep reinforcement learning agents. *IEEE Trans. Softw. Eng.* 49 (7), 3715–3735. <http://dx.doi.org/10.1109/TSE.2023.3269804>.
- Zolfagharian, A., Abdellatif, M., Briand, L.C., S., R., 2024. Safety monitoring of deep reinforcement learning agents. In: *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, ICSE Companion 2024, Lisbon, Portugal, April 14–20, 2024*. ACM, pp. 286–287. <http://dx.doi.org/10.1145/3639478.3643072>.
- Zou, S., Xu, T., Liang, Y., 2019. Finite-sample analysis for SARSA with linear function approximation. In: Wallach, H.M., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E.B., Garnett, R. (Eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada*. pp. 8665–8675, URL: <https://proceedings.neurips.cc/paper/2019/hash/9f9e8cba3700df6a947a8cf91035ab84-Abstract.html>.